Performance-Analyse von Kommunikationsmustern in parallelen Anwendungen auf Basis des Message Passing Interface

Leonard Emter

10. Februar 2026

Version: CleanThesis 0.4.1 - JGU patches

Johannes Gutenberg-Universität Mainz
FB08
Institut für Informatik
High Performance Computing and its Applications

Bachelorarbeit

Performance-Analyse von Kommunikationsmustern in parallelen Anwendungen auf Basis des Message Passing Interface

Leonard Emter

1. Gutachter Prof. Dr. Sarah Neuwirth
Institut für Informatik - High Performance Computing and its Applications
Johannes Gutenberg-Universität Mainz

2. Gutachter Prof. Dr. Michael Wand
Institut für Informatik - Visual Computing
Johannes Gutenberg-Universität Mainz

Betreuer Prof. Dr. Sarah Neuwirth und Niklas Bartelheimer

10. Februar 2026

Leonard Emter
Performance-Analyse von Kommunikationsmustern in parallelen Anwendungen auf Basis des Message Passing Interface
Bachelorarbeit, 10. Februar 2026
Gutachter: Prof. Dr. Sarah Neuwirth und Prof. Dr. Michael Wand
Betreuer: Prof. Dr. Sarah Neuwirth und Niklas Bartelheimer

Johannes Gutenberg-Universität Mainz
High Performance Computing and its Applications
Institut für Informatik
FB08
Staudingerweg 9
55128 Mainz

Zusammenfassung

Diese Arbeit untersucht drei Kommunikationsmuster des Message Passing Interface (MPI) innerhalb einer High Performance Computing (HPC)-Umgebung. Betrachtet werden *non-blocking*, *persistent* und *partitioned* MPI, deren Performance anhand der resultierenden Kommunikationskosten verglichen wird. Die Analyse erfolgt hauptsächlich mit dem *Comb-Benchmark* [@3], der das Kommunikationsverhalten iterativer *Stencil-Berechnungen* abbildet.
Die Ergebnisse verdeutlichen, dass die gemessene Performance stark von der konkreten MPI Implementierung sowie der genutzten HPC-Umgebung beeinflusst werden kann. Diese Einflüsse führen dazu, dass sich in nachrichtengrößenabhängigen Auswertungen ein nahezu inverses Verhalten im Vergleich zur Referenzarbeit [5] zeigt, obwohl die Experimentkonfiguration möglichst nahe an deren Versuchsaufbau angelehnt wurde.
Begleitend werden zwei Micro-Benchmarks ausgewertet, um beobachtete Amortisationseffekte der persistent und partitioned Kommunikationsmuster in iterativen Anwendungen gezielt zu validieren. Dadurch lassen sich die Effekte sowohl in einer isolierten Umgebung reproduzierbar untersuchen als auch konzeptionell nachvollziehbar einordnen.

iv

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation und Problemstellung 1
1.2 Beitrag 2
1.3 Aufbau der Arbeit 3

2 Hintergrund **4**
2.1 Aufbau eines HPC-Systems 4
2.2 Message Passing Interface 5
2.3 MPI Kommunikationsmuster 7
2.3.1 Blocking vs. Non-blocking MPI Kommunikation 7
2.3.2 Persistent MPI Kommunikation 11
2.3.3 Partitioned MPI Kommunikation 12
2.4 Unified Communication X 15
2.5 Stencil-Berechnungen und Halo-Austausch 16

3 Verwandte Arbeiten **20**

4 Design und Methodik **24**
4.1 Comb-Benchmark: Architektur und Erweiterungen 24
4.1.1 Funktionsweise des Halo-Austauschs 24
4.1.2 Partitioned Implementierung 29
4.2 Comb-Studien: Methodik und experimentelle Konfiguration . . . 40
4.2.1 Hardwareumgebung und Software-Stack 40
4.2.2 Parallelisierungsstrategie und Thread-Mapping 41
4.2.3 Parameter für Teststudien 41
4.2.4 Definition der Teststudien 42
4.2.5 Metriken und Bewertungsstrategie 44
4.3 MPI-Ring-Benchmark 46
4.3.1 Setup und Parameterwahl 46
4.3.2 Initialisierung und Topologie 47
4.3.3 Puffer-Management 47
4.3.4 Non-blocking Kommunikation 48
4.3.5 Persistent Kommunikation 48
4.3.6 Messmethodik 49

v

4.3.7 Einschränkung: Partitioned Kommunikation 49
4.4 RDTSC-Benchmark 50
4.4.1 Point-to-Point Management Layer 50
4.4.2 Realisierung des RDTSC-Benchmarks 52

5 Analyse und Auswertung 56
5.1 Comb-Benchmark 56
5.1.1 Strong Scaling Studie 56
5.1.2 Weak Scaling Studie 59
5.1.3 Size-Increase Studie 62
5.1.4 Threads-pro-Prozess Studie 64
5.1.5 Diskussion der Ergebnisse im Kontext der Referenzarbeit 66
5.2 MPI-Ring-Benchmark 69
5.2.1 Absolute Kommunikationszeiten 69
5.2.2 Relative Kommunikationszeiten 70
5.2.3 Validierung der Amortisierungseffekte 71
5.3 RDTSC-Benchmark 72

6 Fazit 73
6.1 Zusammenfassung der Ergebnisse 73
6.2 Ausblick 74

Literatur 76

Abbildungsverzeichnis 80

Tabellenverzeichnis 81

Programmausdrucksverzeichnis 82

1 Einleitung

1.1 Motivation und Problemstellung

Mithilfe von Stencil-Berechnungen kann heute eine Vielzahl mathematischer Probleme gelöst werden, die auch in unserem Alltag eine wichtige Rolle spielen können. Neben dem Lösen partieller Differentialgleichungen wird dieses Konzept beispielsweise dazu eingesetzt, um Luftbewegungen oder Temperatur in der Atmosphäre mathematisch zu simulieren, sodass darauf aufbauend Wettervorhersagen gemacht werden können.
Ein bekanntes Beispiel für ein solches Wettermodell ist ICON (ICOsahedral Nonhydrostatic), das vom Deutschen Wetterdienst und dem Max-Planck-Institut für Meteorologie entwickelt und für globale Wetter- und Klimavorhersagen eingesetzt wird. In ICON werden physikalischen Gleichungen der Atmosphäre auf einem Gitterraum gelöst, wobei Stencil-Operationen zur Berechnung von unter anderem Windgeschwindigkeit und Temperaturen verwendet werden [34].
Im Grunde kann man sich Stencil-Berechnungen so vorstellen, dass ein vorliegendes Problem auf einem Gitter mit einzelnen Zellen abgebildet wird. Jede dieser Zellen des Gitters wird iterativ anhand der Werte ihrer Nachbarzellen aktualisiert, um schrittweise eine numerische Annäherung der gesuchten Lösung zu erhalten.
Daneben gewinnt High Performance Computing (HPC) zunehmend an Bedeutung. Berechnungen komplexer mathematischer Modelle führen dazu, dass der Bedarf an Rechenressourcen stetig weiter ansteigt. Anstatt ein Problem auf einem einzelnen Computersystem zu bearbeiten, das typischerweise nur über eine *Central Processing Unit* (CPU) und *Graphics Processing Unit* (GPU) verfügt, setzt HPC auf den Einsatz vieler leistungsfähiger, miteinander vernetzter Systeme und bündelt deren Rechenleistung, um Aufgaben parallel zu lösen. Dadurch können tausende CPU-Kerne oder mehrere GPUs gleichzeitig an einem Problem arbeiten, um eine Lösung möglichst schnell und effizient zu berechnen.
Kombiniert man nun Stencil-Berechnungen mit HPC, so entsteht ein Ansatz, der es ermöglicht rechenintensive Aufgaben performant in parallelen An-

wendungen zu lösen. In der Praxis wird dazu ein betrachteter Gitterraum in verschiedene Domänen unterteilt. Die dabei entstehenden Subdomänen werden anschließend verschiedenen Prozessen zugeordnet, die diese Teilbereiche des globalen Gitterraums simultan bearbeiten. Tritt nun der Fall ein, dass eine Zelle eines Prozesses innerhalb seiner Domäne Werte benötigt, die von einem anderen Prozess gehalten werden, müssen diese Prozesse miteinander kommunizieren und Daten austauschen, damit das globale Problem korrekt berechnet werden kann. Dieses Kommunikationsverhalten bildet die Grundlage für die Analyse dieser Arbeit.

1.2 Beitrag

Im Mittelpunkt dieser Arbeit stehen somit die Kommunikationskosten, die beim Datenaustausch zwischen Prozessen innerhalb einer parallelen Anwendung entstehen. Dabei werden drei Kommunikationsmuster des Message Passing Interface (MPI) [21] herangezogen und miteinander verglichen. Im Detail werden die Kommunikationsmuster: *non-blocking*, *persistent* und *partitioned* MPI hinsichtlich ihres Performance-Verhaltens in mehreren Teststudien analysiert und beschrieben, wie sich beobachtete Unterschiede erklären lassen. Für den Vergleich wird der sogenannte *Comb-Benchmark* [@3] als Anwendungsszenario eingesetzt, wobei die Arbeit von Collom et al. [5] als Referenz genutzt wird, um erhaltene Ergebnisse in einen Kontext zu setzen. Zusätzlich kommen zwei weitere Benchmarks sowie passende Literatur zum Einsatz, um die Messergebnisse einzuordnen und Ursachen für Performance-Unterschiede zu identifizieren.
Bezüglich des HPC-Aspekts ist es wichtig anzumerken, dass es sich hier um eine CPU-basierte *Punkt-zu-Punkt* Kommunikation handelt, bei der MPI-Prozesse über die jeweils zugeordneten CPU-Kerne Daten miteinander austauschen.

Zusammengefasst sowie auch ergänzend umfasst der Beitrag dieser Arbeit folgende Punkte:

- Implementierung von partitioned MPI innerhalb des Comb-Benchmarks
- Vergleich der MPI-Kommunikationsmuster non-blocking, persistent und partitioned mit Bezugnahme auf die Referenzarbeit [5]
- Entwicklung und Einsatz eigener Benchmarks zur Erklärung beobachtbarer Performance-Unterschiede, darunter ein MPI-Ring-Benchmark

sowie ein RDTSC-Benchmark (Read Time-Stamp Counter) zur Messung von CPU-Taktzyklen in einer vereinfachten Umgebung

1.3 Aufbau der Arbeit

Die nachfolgenden Kapitel beschreiben den weiteren Ablauf der Arbeit. Zunächst werden in Kapitel 2 die grundlegenden Konzepte für diese Arbeit erläutert. Dazu zählen unter anderem Grundlagen zu MPI, die drei relevanten MPI-Kommunikationsmuster und iterative Stencil-Berechnungen. Dieses Kapitel schafft den notwendigen Kontext für die spätere Interpretation der Benchmark-Ergebnisse. Kapitel 3 stell drei verwandte Arbeiten vor, wobei eine davon als Referenzarbeit [5] dient und als Vergleichsbasis für die Konfiguration und Durchführung der Hauptstudien herangezogen wird. In Kapitel 4 werden die drei eingesetzten Benchmarks detailliert beschrieben, einschließlich Funktionsweise, Implementierung und Auswertungsmethodik. Darauf aufbauend präsentiert Kapitel 5 die Ergebnisse der Analyse. Letztendlich schließt Kapitel 6 die Arbeit mit einer Zusammenfassung der zentralen Erkenntnisse ab und beschreibt mögliche Ansätze für weiterführende Arbeiten.

2 Hintergrund

2.1 Aufbau eines HPC-Systems

HPC-Cluster bestehen aus vielen Rechenknoten (*Compute Nodes*), die über ein Hochleistungsnetzwerk miteinander vernetzt sind (z.B. InfiniBand [24]). Jeder Compute Node besitzt mehrere CPU-Kerne und/oder GPUs und kann darüber Prozesse parallel ausführen. Prozesse auf demselben Node kommunizieren über einen gemeinsam genutzten Speicher, was als Intra-Node-Kommunikation bezeichnet wird. Liegen die Prozesse jedoch auf unterschiedlichen Nodes, nutzen sie einen verteilten Speicher, wodurch Nachrichten explizit über das Netzwerk ausgetauscht werden müssen (Inter-Node-Kommunikation). Dieses Zusammenspiel von Kommunikation über geteilten Speicher innerhalb eines Nodes und verteiltem Speicher zwischen Nodes ist typisch für moderne HPC-Systeme.
Neben dem Einsatz von MPI als Kommunikationsmodell in Clustern ist ebenfalls *OpenMP* [6], ein Thread-basiertes Parallelisierungskonzept, in diesem Kontext relevant. Viele Anwendungen, einschließlich des in dieser Arbeit verwendeten Comb-Benchmarks, greifen auf ein hybrides *MPI+OpenMP*-Programmierungsmodell zurück. In diesem Modell übernimmt MPI die Kommunikation über gemeinsamen und verteilten Speicher zwischen den Prozessen, wohingegen OpenMP die Parallelität innerhalb eines Prozesses (ausschließlich über gemeinsamen Speicher) realisiert [26, 33].
HPC-Cluster umfassen nicht nur Compute Nodes, sondern auch spezielle *Login Nodes* und ein oder mehrere *Storage-Server*. Auf dem Login Node können Nutzer sich anmelden, ihre Programme kompilieren, Jobs vorbereiten und Ergebnisse auswerten, während rechenintensive Anwendungen auf den Compute Nodes über einen *Job-Scheduler* (z.B. Slurm [35]) ausgeführt werden. Der Storage-Server wiederum stellt ein verteiltes Dateisystem (z.B. Lustre [2] oder GPFS [29]) über das Netzwerk bereit, das u. a. Ein- und Ausgabedaten speichert [33]. Ein exemplarischer Aufbau eines HPC-Clusters ist in Abbildung 2.1 dargestellt.

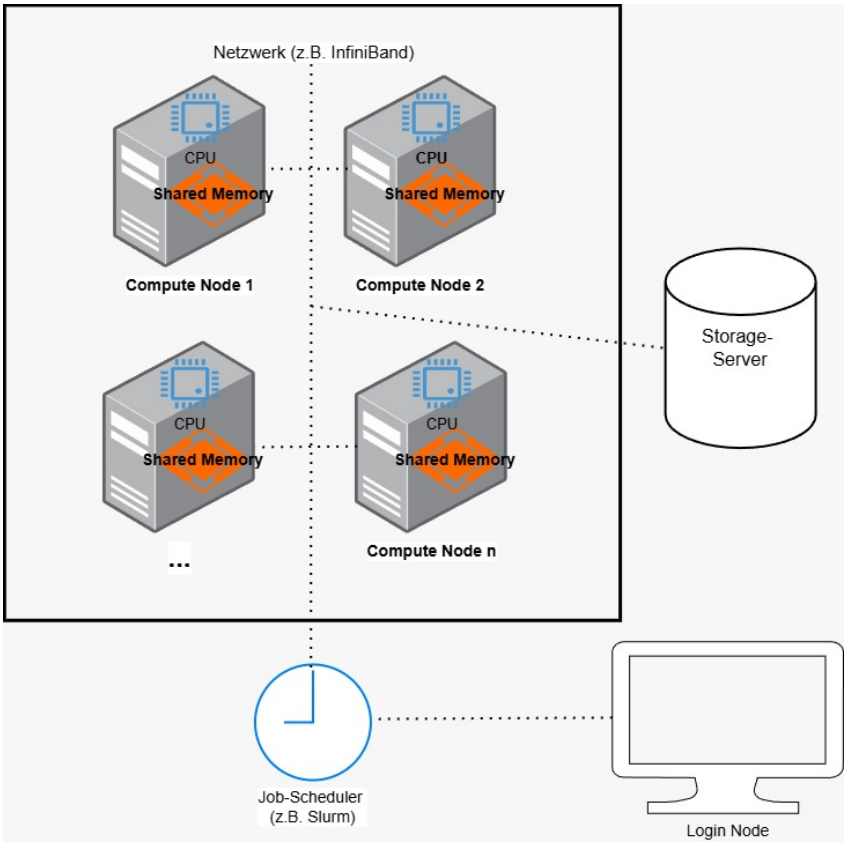


Abb. 2.1: Schematischer Aufbau eines HPC-Clusters

2.2 Message Passing Interface

Das Message Passing Interface (MPI) ist ein standardisierter Ansatz zur Nachrichtenkommunikation für Programme auf Systemen mit verteiltem Speicher. Der Standard definiert eine Menge von Funktionen sowie deren Schnittstellen und Semantiken. Es ist aber weder ein konkretes Netzwerkprotokoll noch eine spezifische Softwarebibliothek, sondern beschreibt lediglich, wie sich eine konforme MPI Implementierung zu verhalten hat [21]. Die eigentliche Umsetzung dieser Konzepte erfolgt dann durch Bibliotheken wie MPICH [10], Open MPI [9] oder Intel MPI [14].
Der MPI-Standard dokumentiert auf einer Übersichtsseite [@20] den Status verschiedener Implementierungen in Bezug auf die jeweiligen MPI-Versionen. Für MPI 4.0 wird dort beispielsweise ausgewiesen, dass MPICH alle im Standard definierten Features vollständig unterstützt, während Open MPI einen weitgehend vollständigen Implementierungsstand dieser Version erreicht hat. Für MPI 3.1 werden mehrere Implementierungen als vollständig konform gelistet, darunter MPICH, Open MPI, Intel MPI, MVAPICH [22] und Cray MPI

[13].
Da in dieser Arbeit die Performance verschiedener Punkt-zu-Punkt Kommunikationsmuster untersucht wird, ist relevant, ab welchen MPI-Versionen die benötigten Funktionen im Standard spezifiziert sind. Non-blocking Punkt-zu-Punkt Kommunikation ist bereits seit frühen MPI-Versionen Teil der Spezifikation, persistent Punkt-zu-Punkt Kommunikation seit Version 1.1, während das partitioned Äquivalent erstmals in MPI 4.0 eingeführt wurde. Open MPI implementiert nach dem offiziellen Status alle Features von MPI 3.1 und nahezu alle Funktionen der Version 4.0, einschließlich der partitioned Variante. Somit sind die drei in dieser Arbeit betrachteten Kommunikationsmuster: non-blocking, persistent und partitioned Punkt-zu-Punkt Kommunikation auf dem verwendeten Cluster (Mogon KI [@36]) unterstützt, da dieser Open MPI als Standardbibliothek zur Verfügung stellt. Die genaue Version dieser Implementierung ist dabei Open MPI 5.0.3.
In der Praxis folgt das MPI Konzept dem *Single Program, Multiple Data* (SPMD)-Paradigma. Es wird zu Programmbeginn eine feste Anzahl paralleler *MPI-Prozesse* erzeugt, die alle dasselbe Programm ausführen, aber auf unterschiedlichen Daten arbeiten. Über ihre eindeutige MPI-Prozess-ID (*Rank*) können diese dann innerhalb der Anwendung unterschieden werden [16]. Mittels des Ranks kann dann im Detail festgelegt werden, welche Prozesse als Sender oder Empfänger agieren und welche Datenpakete sie austauschen. Beispielsweise ruft in einem blocking Punkt-zu-Punkt Muster ein sendender Prozess `MPI_Send` auf, während ein empfangender Prozess `MPI_Recv` verwendet.
MPI unterscheidet grundlegend zwei Klassen der Kommunikation: Punkt-zu-Punkt und kollektive Kommunikation [21]. Bei der ersten Variante tauschen jeweils zwei Prozesse Nachrichten direkt miteinander aus. Auf diesem Mechanismus wird sich im Rahmen dieser Arbeit fokussiert. Die kollektive Kommunikation koordiniert dagegen mehrere MPI-Prozesse gleichzeitig, etwa in Form von globalen Reduktionen. Ein typisches Beispiel dafür ist die Funktion `MPI_Reduce`, welche in dieser Arbeit für die Aggregation von Messdaten (z.B. Minima, Maxima, Summen) eingesetzt wird. Kollektive Operationen werden hierbei aber ausschließlich zur Erfassung bzw. Zusammenführung der Messdaten verwendet und sind nicht Bestandteil der Performance-Analyse der Kommunikationsmuster.
Neben diesen Konzepten spezifiziert MPI unterschiedliche *Threading-Level*, die festlegen, in welchem Umfang mehrere Threads innerhalb eines einzelnen Prozesses MPI-Funktionen aufrufen dürfen. Die Initialisierung einer MPI-Umgebung erfolgt im Code mit der Funktion `MPI_Init_thread` und erlaubt es über den Parameter `required` ein gewünschtes Threading-Level anzufor-

dern. Über den Parameter `provided` wird das von der jeweiligen MPI Implementierung unterstützte Level zurückgegeben. Die folgenden Level sind in der MPI-Spezifikation in aufsteigender Reihenfolge der Thread-Unterstützung definiert [21]:

- `MPI_THREAD_SINGLE`: Nur ein einzelner Thread wird innerhalb eines MPI-Prozesses ausgeführt.
- `MPI_THREAD_FUNNELED`: Ein Prozess kann mehrere Threads besitzen, aber nur der Main-Thread darf MPI-Aufrufe tätigen.
- `MPI_THREAD_SERIALIZED`: Mehrere Threads dürfen im entsprechenden Prozess MPI-Aufrufe tätigen, jedoch nicht gleichzeitig (serialisiert).
- `MPI_THREAD_MULTIPLE`: Mehrere Threads können MPI-Aufrufe ohne Einschränkungen parallel ausführen.

Diese Threading-Level sind insbesondere im Kontext der partitioned Kommunikation relevant, da dort mehrere Threads kooperativ Teilsegmente einer Nachricht übertragen bzw. empfangen können und aufgrund dessen MPI-Aufrufe teilweise gleichzeitig ausführen.

2.3 MPI Kommunikationsmuster

2.3.1 Blocking vs. Non-blocking MPI Kommunikation

Die folgenden Beispiele erläutern die grundlegenden Unterschiede zwischen blocking und non-blocking Punkt-zu-Punkt Kommunikation in MPI anhand einfacher C-Programme. Die Beschreibung der Parameter und die verwendete Terminologie orientieren sich an der MPI-Dokumentation in Version 4.0, sowie an einführenden Übungen der RookieHPC Webseite [21, @28]. Der Beispielcode wurde für diese Arbeit eigenständig erstellt und angepasst, ist jedoch an den genannten Quellen orientiert.

Blocking MPI Send und Receive

Blocking Punkt-zu-Punkt Funktionen wie `MPI_Send` und `MPI_Recv` kehren erst dann zurück, wenn die jeweilige Kommunikationsfunktion abgeschlossen ist und der aufrufende Prozess den zugehörigen Puffer wieder überschreiben

bzw. erneut verwenden darf. Im Fall von `MPI_Send` bedeutet dies, dass die zu sendenden Daten entweder bereits beim Empfänger angekommen oder von der MPI Implementierung in einen internen Puffer kopiert worden sind. Daneben blockiert `MPI_Recv` so lange, bis die erwartete Nachricht vollständig im angegeben Empfangspuffer abgelegt wurde.

```
int rank, size;

// Initialisierung der Umgebung
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

// Sender-Seite
if (rank == 0) {
    int send_value = 42;
    MPI_Send(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

// Empfänger-Seite
} else if (rank == 1) {
    int recv_value;
    MPI_Recv(&recv_value, 1, MPI_INT, 0, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}

// Umgebung "abschließen"
MPI_Finalize();
```

P. 2.1: Blocking `Send/Recv`

Das Code-Beispiel P. 2.1 zeigt eine einfache Kommunikation zweier MPI-Prozesse. Prozess 0 sendet einen Integer-Wert an Prozess 1, welcher diesen über seinen Empfangspuffer verfügbar macht.
Dabei wird die MPI-Umgebung mit `MPI_Init` initialisiert. Anschließend legt `MPI_Comm_size` fest wie viele MPI-Prozesse an der Kommunikation beteiligt sind, wobei `MPI_Comm_rank` jedem ausführenden Prozess eine eindeutige ID (Rank) zuweist.
Das If-Anweisung (rank == 0) beschreibt den “Aufgabenbereich” von Prozess 0. Hier wird der Sendepuffer initalisiert und anschließend der Datentransfer mit `MPI_Send(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD)` angestoßen, wobei die Semantiken der Parameter in Tabelle 2.1 beschrieben sind.

Tab. 2.1: Parameter der Funktion `MPI_Send`

Parameter	Wert	Bedeutung
`buf`	`&send_value`	Adresse des Sendepuffers
`count`	`1`	Anzahl der Elemente
`datatype`	`MPI_INT`	Datentyp jedes gesendeten Elements
`dest`	`1`	Ziel- bzw. Quell-Rank
`tag`	`0`	Nachrichten-*Tag*: Zur Unterscheidung verschiedener Nachrichten
`comm`	`MPI_COMM_WORLD`	Kommunikator: aktuelle MPI-Umgebung/Kontext

Im Pfad des Empfängers (rank == 1) wird mit der komplementären Funktion `MPI_Recv` die Empfangsoperation eingeleitet. Die Parameter sind hierbei äquivalent zu betrachten mit den Ausnahmen, dass hier anstatt des Ziel-Ranks der Quell-Rank angegeben wird und als weiterer Parameter ein `MPI_Status`-Objekt erwartet wird. Dieses zusätzliche Objekt speichert Metadaten über die empfangene Nachricht ab. Hier wird jedoch `MPI_STATUS_IGNORE` übergeben, da die Statusinformationen in diesem Fall nicht benötigt werden. Zum Abschluss rufen alle MPI-Prozesse `MPI_Finalize` auf, um die MPI-Umgebung ordnungsgemäß zu beenden.

Non-blocking MPI Send und Receive

Die non-blocking Operationen wie `MPI_Isend` und `MPI_Irecv` starten ebenfalls eine Kommunikationsoperation, kehren aber sofort zum aufrufenden Programm zurück, ohne dass auf deren Abschluss gewartet werden muss. Der eigentliche Datenaustausch kann dann im Hintergrund weiterlaufen, sodass der Prozess in der Zwischenzeit andere Operationen ausführen kann. Dabei sind die zugehörigen Kommunikationsoperationen über ein `MPI_Request`-Objekt verknüpft.

```
1  int size, rank;
2
3  // Initialisierung der Umgebung
4  MPI_Init(&argc, &argv);
5  MPI_Comm_size(MPI_COMM_WORLD, &size);
6  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
7
8  // Sende-Seite
```

```
9  if (rank == 0) {
10     int send_value = 1337;
11     MPI_Request request;
12     // Daten senden
13     MPI_Isend(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
14     &request);
15     // Warten bis Sendepuffer wieder verfügbar ist
16     MPI_Wait(&request, MPI_STATUS_IGNORE);
17 }
18
19 // Empfangs-Seite
20 if (rank == 1) {
21     int recv_value;
22     MPI_Request request;
23     MPI_Irecv(&recv_value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
24     &request);
25     // Warten bis Daten in Empfangspuffer
26     MPI_Wait(&request, MPI_STATUS_IGNORE);
27 }
28
29 MPI_Finalize();
```

P. 2.2: Non-blocking Send/Recv mit Wait

Das non-blocking Beispiel (P. 2.2) verwendet dieselbe Kommunikationsstruktur, ersetzt aber `MPI_Send/Recv` durch `MPI_Isend/Irecv` und arbeitet mit dem `Request`-Objekt sowie der Funktion `MPI_Wait` zur anschließenden Synchronisierung aller aktiven Kommunikationsvorgänge.
Im Codepfad des Prozesses mit Rank 0 wird nun ein `Request` initialisiert, welches als Argument an `MPI_Isend` übergeben wird, wodurch der Sendevorgang ausgeführt wird. Anschließend erfolgt die Synchronisation des Sendevorgangs über `MPI_Wait`. Im Bereich des Empfängers (Rank 1) wird ein analoges Schema verfolgt.

Letztendlich ist noch anzumerken, dass im betrachteten Code `MPI_Wait` direkt auf `MPI_Isend` bzw. `MPI_Irecv` folgt, sodass sich das effektive Verhalten kaum vom blocking Fall unterscheidet. Der entscheidende Unterschied besteht jedoch darin, dass zwischen dem eigentlichen Kommunikationsaufruf und `MPI_Wait` beliebige weitere Operationen eingefügt werden können, sodass sich die Kommunikation mit anderen Operationen überlappen lässt.

2.3.2 Persistent MPI Kommunikation

In parallelen Programmen werden häufig dieselben Kommunikationsaufrufe mit identischen Argumenten, wie Kommunikationspartner, Datengröße und Datentyp wiederholt in Iterationen aufgerufen, wobei sich lediglich der Dateninhalt ändert. Für solche Szenarien definiert der MPI-Standard 4.0 persistent Punkt-zu-Punkt Operationen, bei denen die Kommunikationsparameter einmalig in einer Kommunikationsanfrage gebunden und anschließend mehrfach wiederverwendet werden. Dadurch wird der Verwaltungsaufwand zwischen MPI-Prozessen und dem Kommunikationssystem reduziert, ohne jedoch den eigentlichen Datentransfer über die Systemhardware zu beschleunigen.

Der Aufbau einer persistent Kommunikation erfolgt dabei in zwei Schritten. Zunächst wird mit den Routinen `MPI_Send_init` und `MPI_Recv_init` ein persistent `Request`-Objekt erstellt. Im Gegensatz zu einem "normalen" `Isend/Irecv`-Request kann ein persistent Request nach `MPI_Wait` wiederverwendet werden. Die Parameter dieser Funktionen entsprechen denen der `Isend/Irecv`-Operationen, jedoch wird kein Datenaustausch gestartet, sondern lediglich initialisiert [21]. In einem zweiten Schritt wird der vorbereitete Request mit dem Befehl `MPI_Start` oder bei mehreren gleichzeitigen Anfragen mit `MPI_Startall` aktiviert. Die Synchronisation innerhalb einer Iteration wird dann wieder mit `MPI_Wait` bzw. `MPI_Waitall` sichergestellt. Nach Abschluss aller Iterationen, also wenn der vollständige Datenaustausch beendet ist, müssen die erstellten Requests mit `MPI_Request_free` explizit freigegeben werden, um die durch die Initialisierung allozierten Ressourcen wieder freizusetzen [21].

Im Folgenden werden nur noch die relevanten Codepfade der einzelnen Prozesse betrachtet. Die MPI-Initialisierung, die Ermittlung von Größe und Rank sowie der Abschluss der Umgebung sind identisch zum vorherigen Codeausschnitt und werden daher nicht mehr dargestellt. Dazu werden Schleifen innerhalb der Anwendung integriert, um die typischen Vorteile von persistent Punkt-zu-Punkt Kommunikation sichtbar zu machen. In P. 2.3 wird ein beispielhafter Ablauf dargestellt.

```
1  if (rank == 0)
2  {
3      int send_value;
4      MPI_Request request;
5
6      // Sende-Operation einmalig initialisieren
```

```
7       MPI_Send_init(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
8       &request);
9
10      for (int i = 0; i < 10; i++)
11      {
12          send_value = 1000 + i;
13          // vorbereitete Operation ausführen + Synchronisation
14          MPI_Start(&request);
15          MPI_Wait(&request, MPI_STATUS_IGNORE);
16      }
17      // Ressourcen wieder freigeben
18      MPI_Request_free(&request);
19  }
20
21  if (rank == 1)
22  {
23      int recv_value;
24      MPI_Request request;
25
26      // Empfangs-Operation einmalig initialsieren
27      MPI_Recv_init(&recv_value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
28      &request);
29
30      for (int i = 0; i < 10; i++)
31      {
32          // vorbereite Operation ausführen + Synchronisation
33          MPI_Start(&request);
34          MPI_Wait(&request, MPI_STATUS_IGNORE);
35      }
36      // Ressourcen wieder freigeben
37      MPI_Request_free(&request);
38  }
```

P. 2.3: Persistent MPI mit Schleifen

2.3.3 Partitioned MPI Kommunikation

Der MPI-Standard 4.0 beschreibt partitioned Punkt-zu-Punkt Kommunikation als Erweiterung der persistent Variante. Auch hier werden Kommunikationsargumente per `MPI_Psend_init` bzw. `MPI_Precv_init` einmalig gebunden. Zusätzlich werden nun aber die Nachrichtenpuffer in mehrere Teilsegmente aufgeteilt, die in der MPI-Terminologie als *Partitionen* bezeichnet werden. Diese dürfen dabei von unterschiedlichen Ausführungseinheiten, etwa verschiedenen *Threads* innerhalb desselben MPI-Prozesses, bearbeitet werden.

Auf der Senderseite kann dann mit der Funktion `MPI_Pready` eine abgeschlossene Partition als “bereit” gemeldet werden, was den Datentransfer zusätzlich parallelisieren kann. Es ist aber anzumerken, dass in der MPI-Dokumentation angegeben wird, dass die jeweiligen MPI Implementierungen “frei sind, zu entscheiden”, wann und wie viele Partitionen übertragen werden sollen, sobald sie als “bereit” markiert worden sind [21].

```
const int num_partitions = 2;
int send_data[num_partitions];
int recv_data[num_partitions];
MPI_Request send_request, recv_request;

// Initialisierung der persistent Requests
if (rank == 0)
    MPI_Psend_init(send_data, num_partitions, 1, MPI_INT, 1, 0,
                   MPI_COMM_WORLD, MPI_INFO_NULL, &send_request);
if (rank == 1)
    MPI_Precv_init(recv_data, num_partitions, 1, MPI_INT, 0, 0,
                   MPI_COMM_WORLD, MPI_INFO_NULL, &recv_request);



for (int iter = 0; iter < 10; iter++)
{
    if (rank == 0)
    {
        // Datentransfer einleiten
        MPI_Start(&send_request);

        // Parallel Pufferbefüllen
#pragma omp parallel num_threads(num_partitions)
        {
            int tid = omp_get_thread_num();
            send_data[tid] = 100 + tid;

            // Partition "tid" ist nun fertig
            MPI_Pready(tid, send_request);
        }

        MPI_Wait(&send_request, MPI_STATUS_IGNORE);
    }

    if (rank == 1)
    {
        // Daten erhalten. Hier ohne explizite Parallelisierung
        MPI_Start(&recv_request);
        MPI_Wait(&recv_request, MPI_STATUS_IGNORE);
```

```
41        }
42    }
43    // Request freigeben
44    if (rank == 0)
45        MPI_Request_free(&send_request);
46    if (rank == 1)
47        MPI_Request_free(&recv_request);
```

P. 2.4: Partitioned MPI mit MPI_Init_thread und OpenMP-Threads

In P. 2.4 wird zusätzlich `<omp.h>` eingebunden, wodurch die OpenMP-API für die Thread-basierte Parallelisierung innerhalb eines MPI-Prozesses verfügbar gemacht wird. Wichtig zu betonen ist, dass das Threading-Level innerhalb von MPI miteinzubeziehen ist. Anstatt die Umgebung mit `MPI_Init` zu initialisieren, wird `MPI_Init_thread` mit dem Argument `MPI_THREAD_MULTIPLE` genutzt. Damit wird ermöglicht, dass Threads innerhalb eines Prozesses uneingeschränkt MPI-Funktionen aufrufen dürfen (vgl. Abschnitt 2.2).
In der Hauptschleife nutzt Prozess 0 eine OpenMP-Parallelregion (`#pragma omp parallel`), um für jede Partition einen eigenen Thread zu starten. Nach dem Start des vorbereiteten Requests werden in dieser Parallelregion alle Partitionen mit `MPI_Pready` als sendebereit markiert, sobald diese ihre interne Arbeit abgeschlossen haben.
Die Empfängerseite verzichtet in diesem Beispiel auf Thread-Parallelisierung. Hier könnte zusätzlich der Einsatz von `MPI_Parrived` eingebunden werden, um empfangene Teilnachrichten bereits verfügbar zu machen, sobald sie beim Empfänger eintreffen, und diese direkt in Folgeberechnungen zu integrieren. Da die Referenzarbeit [5] jedoch bewusst darauf verzichtet, `MPI_Parrived` einzusetzen, wird im Folgenden nicht weiter darauf eingegangen.
Letztendlich aktiviert Prozess 1 den vorbereiteten Request, wartet auf den vollständigen Empfang aller Partitionen und anschließend werden alle erstellten Requests wieder freigegeben.

Ergänzend ist zu erwähnen, dass der Zusammenhang von MPI und Threads hier exemplarisch im Kontext der partitioned Kommunikation dargestellt wurde. Threads dürfen jedoch ebenso in den non-blocking und persistent Varianten eingesetzt werden, können aber nicht an dem Datentransfer mitwirken.
Um den Unterschied der Thread-Parallelisierung kenntlich zu machen, wird Abbildung 2.2 herangezogen. Hier zeigt sich der zeitliche Ablauf eines traditionellen Sendevorgangs im Vergleich zu der partitioned Variante, wobei vier OpenMP-Threads in allen Kommunikationsvarianten zum Einsatz kommen.

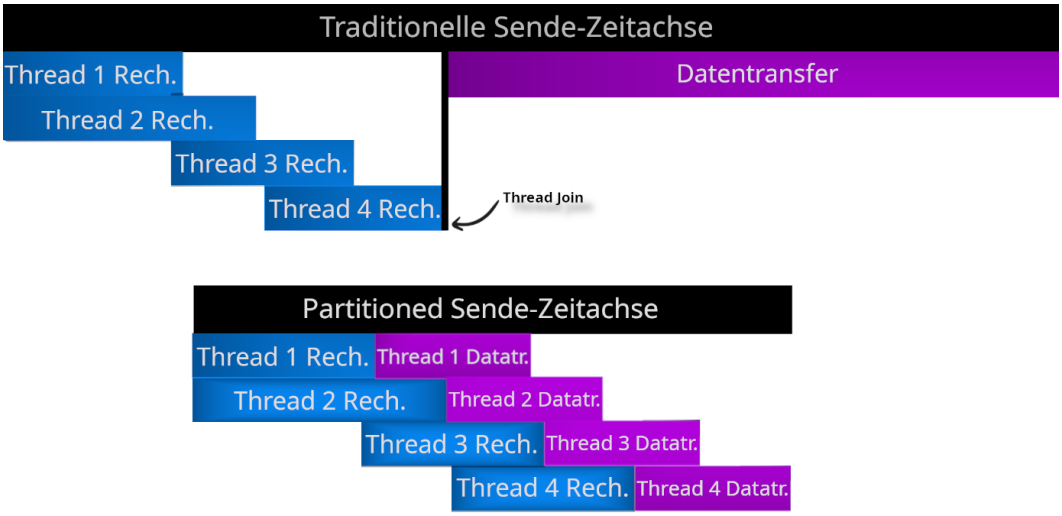


Abb. 2.2: traditioneller vs. partitioned Sendeverlauf (angelehnt an [12])

In der oberen traditionellen Sende-Zeitachse entspricht der Ablauf dem Muster non-blocking oder persistent Kommunikation. Mehrere Threads packen ihre Daten in einen gemeinsamen Sendepuffer und erst nachdem der letzte Thread seine Arbeit abgeschlossen hat und alle Threads zusammengeführt wurden (*Thread Join*), wird der gesamte Puffer als eine zusammenhängende Nachricht übertragen.
In der Sende-Zeitachse von partitioned MPI ist zu erkennen, dass ein Thread seine Partition unmittelbar nach Abschluss seiner Berechnung überträgt, statt auf die Fertigstellung aller anderen Threads zu warten. Dadurch können Berechnung und Kommunikation stärker überlappen. Gleichzeitig verteilt sich die Übertragung zeitlich, sodass nicht ein einzelner gebündelter Datenburst entsteht, sondern mehrere kleinere Sendungen überlappend stattfinden.

2.4 Unified Communication X

Unified Communication X (UCX) ist ein Kommunikationsframework, das eine einheitliche Schnittstelle für unterschiedliche Netzwerk- und Speichertechnologien (Hardwareplattformen) bereitstellt. Damit dient es als gemeinsame Vermittlungsschicht für verschiedene Programmiermodelle, darunter auch für das in dieser Arbeit relevante MPI. Dabei adressiert es das Problem, dass klassische MPI Implementierungen für jede Hardwareplattform separate Transportmodule benötigen. Zu diesen Plattformen gehören unter anderem

Hochleistungsnetzwerke wie InfiniBand oder auch geteilte Speichermechanismen. UCX fungiert somit als gemeinsame Kommunikationsschicht zwischen der MPI Implementierung und der zugrunde liegenden Hardware.

Die Architektur von UCX lässt sich in drei Ebenen unterteilen: *UC-Protocols* (UCP), *UC-Transports* (UCT) sowie *UC-Services* (UCS).
UCP stellt High-Level-Kommunikationsfunktionen bereit, die für die Programmiermodelle relevant sind. Dazu gehören Remote Memory Access (RMA), atomare Operationen, Tag-Matching, Active Messages und einfache Kollektivoperationen. Darüber hinaus übernimmt UCP Aufgaben wie die Auswahl geeigneter Transportschichten und die Fragmentierung von Datenpaketen.
Auf der darunterliegenden UCT-Ebene werden die eigentlichen Kommunikationsvorgänge gekapselt. Sie stellt Low-Level-Operationen bereit, die auf unterschiedlichen Treibern aufsetzen, ohne dass die oberen Schichten die Details kennen müssen.
Die unterste Ebene UCS stellt schließlich allgemeine Hilfsoperationen für Thread-Sicherheit, Speicherpools und grundlegende Datenstrukturen bereit, die von UCT und UCP genutzt werden.

UCX ist zusammengefasst ein Framework, welches MPI die Arbeit abnimmt, für verschiedene Hardwareplattformen eigene Low-Level-Schnittstellen zu implementieren. Damit ist die MPI Implementierung in diesem Kontext für die Programmierlogik verantwortlich, während UCX die konkrete Datenübertragung auf der Hardware des Systems übernimmt [31].

2.5 Stencil-Berechnungen und Halo-Austausch

Viele numerische Simulationscodes verwenden *Stencil-Operationen*, bei denen der Wert eines Gitterpunktes aus den Werten einer festen Menge benachbarter Punkte bestimmt wird. Anschaulich lässt sich dies als zweidimensionales Rechteck beschreiben, das in kleinere Rechtecke (Gitterzellen) unterteilt ist. Jede Zelle speichert dabei bestimmte Daten. In Stencil-Anwendungen werden diese dann iterativ aktualisiert, sodass sich eine Lösung schrittweise ergibt. Die Aktualisierungen der einzelnen Zellenwerte beruhen auf einem *Stencil-Operator*, der das Nachbarschaftsschema für alle Zellen definiert. Hierbei ist wichtig Stencil-Operatoren von Stencil-Operationen abzugrenzen. Operatoren beschreiben in diesem Kontext ausschließlich das Nachbarschaftsschema,

während Operationen konkrete Rechnungen auf diesen Operatoren beschreiben.

	-1	
-1	4	-1
	-1	

Abb. 2.3: 5-Punkt-Stencil [17]

Ein klassisches Beispiel für einen Stencil-Operator ist der *5-Punkt-Stencil*, wie in Abbildung 2.3 zu sehen ist. In diesem Beispiel ist die Zelle mit dem Wert 4 die Zelle, die es zu aktualisieren gilt. Der dargestellte Stencil beschreibt die Nachbarschaft so, dass alle vertikalen und horizontalen Nachbarn relevant sind, einschließlich des Werts der zentralen Zelle selbst. Um eine konkrete Aktualisierung durchzuführen, muss auf der Nachbarschaft eine Rechenvorschrift angewendet werden. Eine einfache Vorschrift, um die Zelle konkret zu aktualisieren, könnte etwa sein, alle fünf Werte zu addieren und daraus das arithmetische Mittel zu bilden.

Für eine parallele Ausführung wird das globale Gitter per *Domänendekomposition* in rechteckige Subdomänen zerlegt, die zu einem kartesischen Prozessgitter zusammengesetzt werden. Jede dieser Subdomänen wird also von einem Prozess verwaltet, der die zugehörigen Zellenwerte lokal hält.
Nun tritt das Problem auf, dass für die Aktualisierung mancher Zellen innerhalb einer lokalen Domäne Werte einer anderen Subdomäne benötigt werden. Hier setzt der MPI-Datenaustausch an. Jede lokale Domäne erweitert ihr Randgebiet um *Halo-Zellen*, in denen die benötigten Nachbarwerte gespeichert werden. In jeder Iteration werden diese Halo-Zellen per Punkt-zu-Punkt Kommunikation mit den benachbarten Prozessen ausgetauscht und anschließend kann jeder Prozess seine lokalen Zellenwerte berechnen.

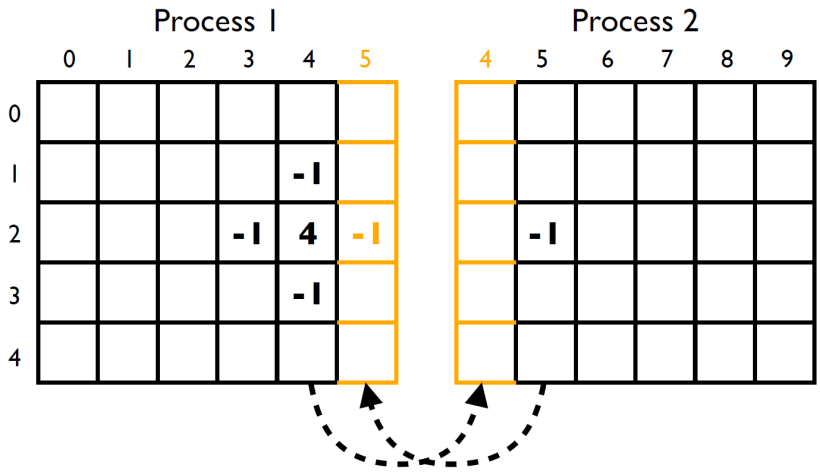


Abb. 2.4: Halo-Austausch zweier Subdomänen 5-Punkt-Stencil [17]

In Abbildung 2.4 wird visuell dargestellt, weshalb die Kommunikation verschiedener Prozesse notwendig ist, um eine 5-Punkt-Stencil-Operation auszuführen. Zelle (4,2) mit dem Wert 4 benötigt neben ihren lokalen Nachbarzellen eine Zelle, die innerhalb der Subdomäne von Prozess 2 liegt. Daher wird vor der Aktualisierung der Gitterzellen von Prozess 1 die notwendige Spalte des Nachbarprozesses übertragen. Analog gilt dies auch für Prozess 2. Konkret wird also Spalte 5 an Prozess 1 übertragen, welche als Halo-Spalte an diesen angefügt wird und Prozess 2 erhält dementsprechend Spalte 4. Auf diese Weise ist sichergestellt, dass jeder Prozess die benötigten Nachbarwerte speichert, um die Stencil-Operation für seine lokalen Zellen auszuführen.

Je nach Stencil-Operator erweitert sich die Menge der benötigten Nachbarzellen. Während im 5-Punkt-Stencil nur die direkten Nachbarn entlang der Koordinatenachsen beteiligt sind, betrachtet der *9-Punkt-Stencil* zusätzlich die vier Diagonalnachbarn einer Zelle, sodass neben Kanten- auch Eckzellen mit benachbarten Prozessen ausgetauscht werden.

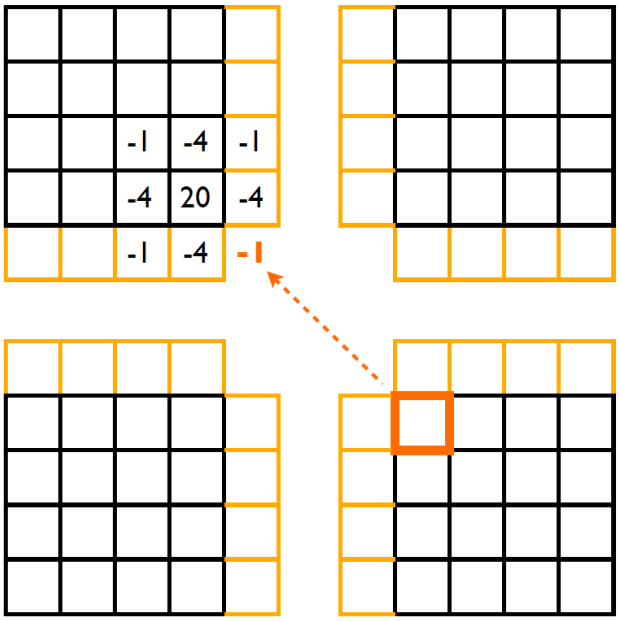


Abb. 2.5: Halo-Austausch für den 9-Punkt-Stencil [17]

Wie in Abbildung 2.5 gezeigt, sind vier Prozessdomänen zu sehen. Durch die Aufteilung auf vier Prozesse werden nicht nur Zellen gemeinsamer Spalten, sondern auch Zeilen benachbarter Prozesse benötigt. Hinzu kommt aufgrund des 9-Punkt-Stencils, dass der oben links dargestellte Prozess den Eckpunkt der diagonalen Subdomäne benötigt. Aus diesem Grund muss ebenfalls dieser Wert ausgetauscht und innerhalb des Prozesses als Halo-Zelle abgespeichert werden [17, 5].

Erweitert man den 9-Punkt-Stencil nun auf drei Dimensionen, so ergibt sich der *27-Punkt-Stencil*. Für die Aktualisierung jedes Zellenwerts werden die Werte von 26 Nachbarpunkten sowie der eigene benötigt [5]. Visuell lässt sich dies mit einem Rubik's Cube veranschaulichen. Der zu aktualisierende Wert entspricht dem inneren Kern, während die 26 äußeren kleinen Würfel die Nachbarzellen repräsentieren. Aufgrund dieser Nachbarschaft lässt sich für die benötigten Halo-Bereiche Folgendes ableiten: In einem großen zusammengesetzten Prozessgitter brauchen Prozesse dieser Zusammensetzung 6 Flächen-, 12 Kanten und 8 Eck-Halo-Bereiche benachbarter Prozesse. Entsprechend müssen, wie Collom et al. [5] es beschreiben, 26 Halo-Nachrichten gesendet und empfangen werden. Genau dieses Muster des Datenaustauschs verfolgt der Comb-Benchmark. In Unterunterabschnitt 4.1.1 folgen visuelle Darstellungen bezüglich dieses Austauschmusters.

3 Verwandte Arbeiten

Temuçin et al. [12] untersuchen partitioned MPI und sprechen ein zentrales Problem an. Die potenziellen Vorteile dessen lassen sich mit klassischen MPI-Benchmarks nur schwer erfassen. Als Beitrag stellen die Autoren die erste öffentlich verfügbare Micro-Benchmark-Sammlung vor, die gezielt auf die Eigenschaften von partitioned MPI ausgelegt ist.
Der Kernvorteil von partitioned MPI in *multithreaded-Szenarien* liegt darin, dass Datenübertragungen nicht erst nach einem globalen Thread-Join starten, sondern bereits dann, wenn einzelne Threads ihre jeweiligen Partition fertiggestellt haben. So kann die Kommunikation zwischen Prozessen früher beginnen und mit parallel laufenden Berechnungen überlappt werden können. Ihre Ergebnisse zeigen, dass dieser Ansatz vor allem in realistischen HPC-Umgebungen mit Systemrauschen und Lastungleichgewichten Vorteile bietet.
Um diese Effekte zu messen, führen sie neue Metriken ein:

- **Overhead:** Die zusätzliche Last, die durch das Aufteilen einer Nachricht in mehrere Partitionen im Vergleich zu einem einzelnen Standard Sendeaufruf entsteht.
- **Perceived Bandwidth:** Die effektive Datenrate aus Sicht der Anwendung, basierend auf der Zeitspanne vom Start des Datentransfers der letzten Partition bis zum vollständigen Erhalt der Nachricht.
- **Application Availability:** Die CPU-Zeit, die der Anwendung zusätzlich zur Verfügung steht, weil Kommunikation im Hintergrund ausgeführt wird.
- **Early-Bird Communication:** Der Anteil der Daten, der bereits übertragen wurde, bevor ein äquivalenter Thread-Join im klassischen Kommunikationsmuster stattgefunden hätte.

Die Metriken werden anhand isolierter Punkt-zu-Punkt-Kommunikation, Sweep-Kommunikation und einem Halo-Austausch (7-Punkt-Stencil) ausgewertet. Die Autoren variieren künstliches Systemrauschen, Cache-Zustände (Hot vs.

20

Cold Cache) und die Anzahl der Partitionen. Die Auswertung zeigt, dass partitioned MPI vor allem dann von Vorteil ist, wenn Kommunikation zeitlich mit Berechnungen überlappen kann und Unterschiede in der Laufzeit einzelner Threads auftreten. So lassen sich Synchronisationskosten reduzieren, weil ein expliziter Thread-Join bei der optimierten Variante nicht notwendig ist. Für Latenz-dominierte Kommunikationsphasen oder sehr kleine Nachrichten gibt es nur geringe oder keine Vorteile gegenüber klassischen MPI-Ansätzen. In Sweep-Kommunikations-Szenarien beobachten sie unter vorteilhaften Bedingungen einen bis zu 15-fach höheren Durchsatz gegenüber Standard MPI. Gleichzeitig machen die Experimente deutlich, dass die Partitionsanzahl an die Hardware angepasst werden muss. Überschreitet sie jedoch die Anzahl physischer Kerne eines Sockets auf einem Node, treten spürbare Leistungseinbrüche auf.
Für die Halo-Kommunikation fallen die beobachteten Unterschiede zwischen partitioned und klassischen MPI-Verfahren vergleichsweise gering aus.
Insgesamt liefert die Arbeit keine allgemeine Aussage zur Leistungsfähigkeit, sondern eine alternative Methodik zur Bewertung von partitioned MPI. Sie zeigt, unter welchen Bedingungen es Vorteile mit sich bringt und bildet eine wichtige Grundlage für die praktische Nutzung und Weiterentwicklung der partitioned Variante.

Die Arbeit von Schneewind und Niethammer [30] behandelt die Implementierung von partitioned MPI in Open MPI 5.0.3 und bewertet, inwieweit sich Performance-Vorteile gegenüber klassischen MPI Kommunikationsmustern ergeben.
Dazu entwickelten die Autoren einen Micro-Benchmark, der partitioned MPI mit alternativen Ansätzen wie single-threaded blocking und persistent MPI sowie deren multithreaded-Varianten vergleicht. Der Fokus liegt aber primär auf partitioned MPI. Neben einer herkömmlichen Variation der Partitionszahl, verändern sie ebenfalls das Schema, in dem Partitionen mit `Pready` freigegeben werden (sequentiell, randomisiert, Halo-ähnlich).
Die Messungen zeigen, dass in Open MPI die partitioned Kommunikation in multithreaded-Szenarien bessere Performance als persistent MPI erreichen kann, insbesondere wenn viele Threads beteiligt sind. Gleichzeitig wird deutlich, dass die partitioned MPI in Open MPI intern im Wesentlichen auf persistent Requests abgebildet wird, wodurch die theoretischen Vorteile in manchen Szenarien begrenzt bleiben.
Ein weiteres zentrales Ergebnis ist, dass die Performance bei sehr feingranularen Partitionierungen deutlich einbricht. Werden viele kleine Partitionen

verwendet, dominiert der Verwaltungs- und Nachrichten-Overhead. Um dieses Problem zu lösen, schlagen die Autoren eine Aggregationsstrategie vor, bei der mehrere vom Anwender definierte Partitionen intern zu größeren Übertragungseinheiten zusammengefasst werden, bevor gesendet wird. Erste Prototyp-Studien zeigen, dass sich vor allem für kleine Partitionen bessere Ergebnisse erzielen lassen, sofern die Partitionen in einer günstigen Reihenfolge freigegeben werden. Randomisierte Freigabemuster profitieren jedoch weniger von dieser Optimierung.
Letztendlich kommt die Arbeit zu dem Schluss, dass partitioned MPI in Open MPI bereits Potenzial für Performance-Verbesserungen bietet, die aktuelle Implementierung jedoch noch nicht optimal mit stark fragmentierten Nachrichten umgeht. Die Arbeit ist daher weniger als abschließende Bewertung zu verstehen, sondern als Ausgangspunkt für gezielte Optimierungen.

Collom et al. [5] betrachten die Performance von non-blocking, persistent und partitioned MPI speziell für Stencil-Berechnungen. Da diese Berechnungen auf iterativen Halo-Austauschen basieren, stellen sie ein ideales Anwendungsszenario dar, um die Initialisierungskosten der persistent Kommunikation über viele Iterationen hinweg zu amortisieren. Zur Evaluation integrierten die Autoren persistent und partitioned MPI in den Comb-Benchmark des Lawrence Livermore National Laboratory (LLNL) [@3], wobei OpenMP-Threads für das *Packing/Unpacking* der Daten in zusammenhängende Puffer genutzt werden.
Die auf Basis von MVAPICH2 (2.3.7) durchgeführten Messungen liefern folgende zentrale Ergebnisse:

- **Größte Speedups:** Während persistent MPI Ergebnisse von bis zu 37 % erzielt, erreicht partitioned MPI Speedups (gegenüber non-blocking) von bis zu 68 % in der Strong Scaling Studie.

- **Abhängigkeit der Nachrichtengröße:** Der Nutzen ist stark an die Datenmenge gekoppelt. Bei großen Nachrichten (ab 65536 Gitterzellen) überholt das partitioned Muster die Baseline deutlich. Bei sehr kleinen Nachrichten (256 Gitterzellen) verursacht der Verwaltungs-Overhead jedoch einen massiven Performance-Rückgang von bis zu 73 % gegenüber dem non-blocking MPI.

- **Einfluss der Thread-pro-Prozess-Aufteilung:** Die besten Ergebnisse dieser Studie werden bei einer moderaten Thread-Anzahl pro Prozess erzielt (8 Threads pro Prozess). Eine Konfiguration mit 64 Threads pro

Prozess führt aufgrund des steigenden Overheads für das Partitionsmanagement zu einem Einbruch unter dem non-blocking Niveau.

Sie sprechen davon, dass das volle Potenzial von partitioned MPI in dieser Untersuchung noch nicht vollständig ausgeschöpft wurde, da die Empfängerseite aktuell auf die Fertigstellung aller Partitionen wartet, anstatt einzelne Segmente sofort über `Parrived` zu verarbeiten. An dieser Stelle sei jedoch angemerkt, dass die Einbindung von `Parrived` in Comb nur begrenzt zielführend ist. Da eine Iteration erst nach vollständigem Abschluss fortgesetzt werden kann und keine Folgeberechnungen modelliert werden, würde `Parrived` in diesem Benchmark voraussichtlich ausschließlich zusätzlichen Overhead erzeugen. Dennoch belegt die Studie, dass persistent und partitioned MPI gerade für iterative Stencil-Anwendungen ein erhebliches Potenzial bietet und eine effiziente Alternative zu klassischen Kommunikationsmustern darstellt. In Anlehnung an Collom et al. nutzt diese Arbeit denselben Benchmark und orientiert sich dabei an deren Problemgrößen und Prozesszahlen, um Vergleichbarkeit zu gewährleisten.

Zusammenfassende Betrachtung der verwandten Arbeiten

Die untersuchten Arbeiten zeigen eine klare Entwicklung in der Bewertung moderner Punkt-zu-Punkt Kommunikationsmuster des Message Passing Interface. Temuçin et al. [12] etablieren grundlegende Metriken und zeigen die Leistungscharakteristiken unter Systemrauschen auf. Daneben identifizieren Schneewind und Niethammer [30] durch ihre Implementierungsanalyse die Aggregation als notwendige Optimierung für feingranulare Partitionierungen. Zuletzt demonstrieren Collom et al. [5], dass partitioned gegenüber non-blocking und persistent MPI erhebliche Vorteile für reale wissenschaftliche Anwendungen bieten kann.

4 Design und Methodik

4.1 Comb-Benchmark: Architektur und Erweiterungen

Comb ist ein von *Lawrence Livermore National Laboratory* (LLNL) entwickeltes Benchmark-Tool, das speziell für Stencil-Anwendungen entworfen wurde [@3]. Im Folgenden wird die Halo-Kommunikation dieses Benchmarks beschrieben. Zunächst werden dafür das explizite *Packing* und *Unpacking* erläutert sowie die geometrische Zusammensetzung des Nachrichtenaustauschs hergeleitet. Anschließend erfolgt eine Validierung mithilfe eines MPI-Profilers, bevor die Implementierung der partitioned Variante vorgestellt wird.

4.1.1 Funktionsweise des Halo-Austauschs

Der Comb-Benchmark bildet den in Kapitel 2 beschriebenen Halo-Austausch im dreidimensionalen Fall mit einem 27-Punkt-Stencil nach und erlaubt es, verschiedene MPI Kommunikationsmuster unter kontrollierten Bedingungen zu untersuchen. Um mit den teilweise verstreuten Halo-Daten korrekt umzugehen wird dabei ein bestimmte Herangehensweise verfolgt.

Explizites Packing & Unpacking

Beim Halo-Austausch müssen sowohl zusammenhängende als auch nicht zusammenhängende Randbereiche im Speicher behandeln werden, weshalb konsequent auf ein explizites Packing und Unpacking Verhalten gesetzt wird. In einem zeilenweisen Speicherlayout einer 3D-Domäne liegen bestimmte Flächen bereits zusammenhängend im Speicher, exemplarisch die obere und untere Fläche eines lokalen Gitters. Seitliche Flächen oder Kanten bestehen dagegen aus vielen kleinen, über den Speicher verteilten Segmenten und sind damit nicht zusammenhängend. Anstatt diese nicht zusammenhängenden

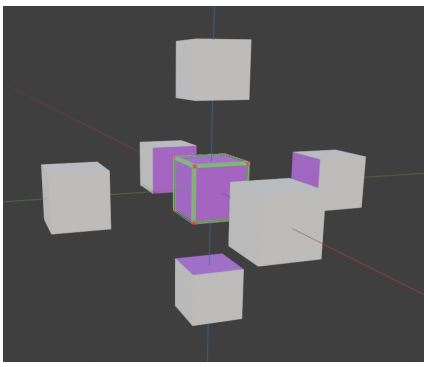
24

Bereiche direkt mit komplexen MPI-Datentypen zu übertragen, führt die Referenzarbeit [5] alle für den Halo-Austausch benötigten Zellen in ihren Studien in einem zusammenhängenden Kommunikationspuffer zusammen.
Vor jedem Austausch werden die benötigten Randdaten aus dem lokalen Gitter in einen zusammenhängenden Puffer kopiert. Dieses Packing wird mit OpenMP-Threads parallelisiert, sodass mehrere Kerne gleichzeitig an der Vorbereitung der Halo-Daten arbeiten können. Aus Sicht von MPI werden anschließend nur noch zusammenhängende Nachrichten verschickt, was die Kommunikation vereinfacht.
Nach dem Empfang der Nachrichten mit den Randdaten der Nachbarprozesse werden die empfangenen Puffer wieder in die lokale Gitterstruktur zurückgeschrieben. Dabei werden die Werte aus den zusammenhängenden Empfangspuffern an die richtigen Positionen der Halo-Zellen im lokalen Gitter kopiert, sodass die Stencil-Anwendung in der nächsten Iteration wieder auf konsistente Nachbarinformationen zugreifen kann. Ein vollständiger Schritt im Comb-Benchmark besteht damit immer aus dem Packing der Randzellen, Austausch der Nachrichten und anschließendem Unpacking in die Halo-Bereiche [5]. Dieses Muster bildet eine Grundlage für die eigene partitioned Implementierung in Unterabschnitt 4.1.2.

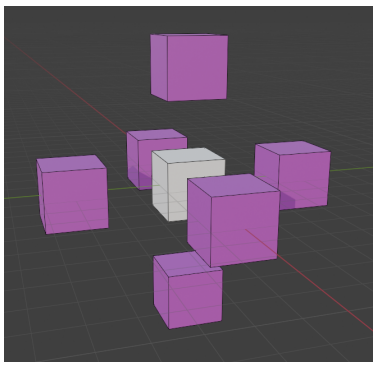
Geometrische Herleitung des Halo-Austauschs

Um den Nachrichtenaustausch in Comb anschaulich zu machen kann, man sich eine $3 \times 3 \times 3$ Anordnung von Würfel vorstellen, also 27 Prozesse, die jeweils eine lokale Subdomäne repräsentieren. Im Fokus steht dabei der zentrale Würfel, dessen Datentransfer an seine 26 Nachbarn betrachtet wird. Die sechs Flächen dieses Würfels werden in der Darstellung violett markiert, die zwölf Kanten grün und die acht Eckpunkte rot. Jeder dargestellte Würfel steht für einen Prozess, der einen bestimmten Teil des globalen Gitters abdeckt. Die lokalen Gitterzellen werden zur Vereinfachung nicht mit dargestellt.

Abbildung 4.1 visualisiert zunächst den Austausch der Flächen. Der zentrale Prozess überträgt insgesamt sechs Halo-Flächen an die direkt benachbarten Prozesse. Die jeweiligen Flächengrößen in Bezug darauf sind in Tabelle 4.1 angegeben.



(a) detaillierte Darstellung der sechs Flächennachbarn



(b) vereinfachte Darstellung der sechs Flächennachbarn

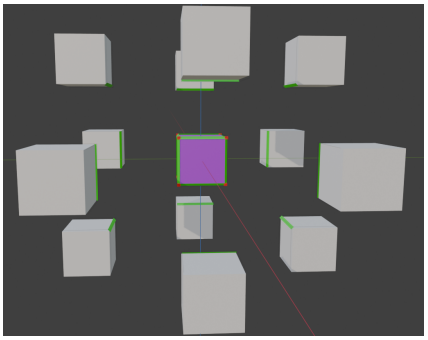
Abb. 4.1: Comb: Flächenaustausch

Richtung	Anzahl	Größe (Zellen)
Oben/Unten	2	$x \times y$
Vorne/Hinten	2	$x \times z$
Links/Rechts	2	$y \times z$

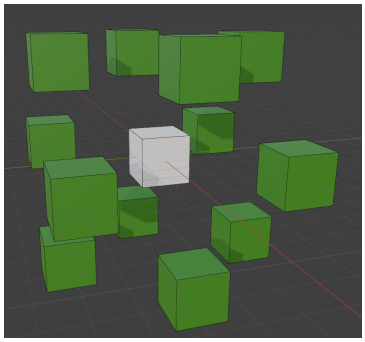
Tab. 4.1: Comb: geometrische Zusammensetzung der Halo-Flächen

Dabei bezeichnen x, y und z die Anzahl der Zellen des lokalen Gitters der Prozesses in der jeweiligen Dimensionsrichtung.

In Abbildung 4.2 wird die Kantenübertragung dargestellt. Hier werden die Randzellen entlang der Kanten des zentralen Würfels an die Nachbarprozesse geschickt. Insgesamt entstehen dabei jeweils vier Kanten der der Länge x, y und z, entsprechend der Dimensionen des lokalen Prozessgitters.



(a) detaillierte Darstellung der zwölf Kantennachbarn

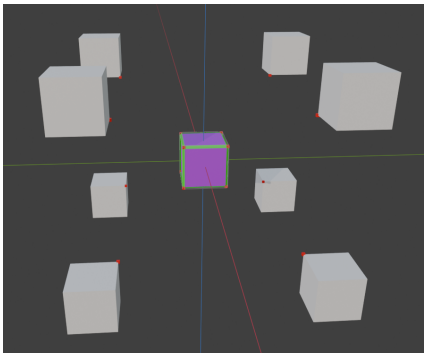


(b) vereinfachte Darstellung der zwölf Kantennachbarn

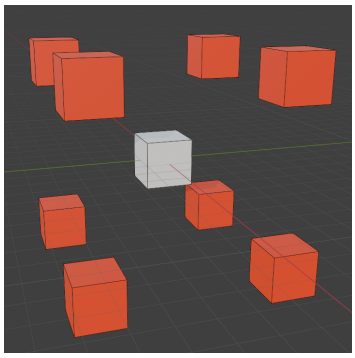
Abb. 4.2: Comb: Kantenaustausch

Abbildung 4.3 zeigt den Austausch der Eckpunkte. In diesem Schritt geht es um die acht Nachbarprozesse, die nur an einer Ecke des zentralen Würfel angrenzen. An jeden dieser Ecknachbarn wird genau eine einzelne Zelle

versendet, sodass insgesamt acht Eckzellen von diesem Prozess an seine acht Nachbar verschickt werden.



(a) detaillierte Darstellung der acht Eckpunktnachbarn

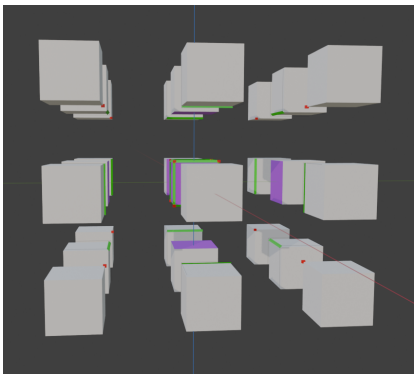


(b) vereinfachte Darstellung der acht Eckpunktnachbarn

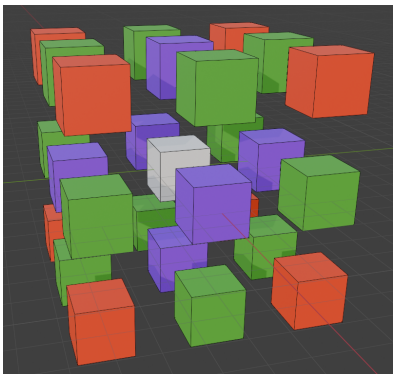
Abb. 4.3: Comb: Eckpunktaustausch

In Abbildung 4.4 ist dargestellt, wie Halo-Nachrichten im Comb-Benchmark geometrisch ausgetauscht werden. Dabei gilt, dass jeder Prozess 26 Halo-Bereiche sendet und empfängt. Diese bestehen aus:

- 6 Flächen der Größe: $2xy + 2xz + 2yz$ Zellen
- 12 Kanten der Größe: $4x + 4y + 4z$ Zellen
- 8 Eckpunkte der Größe: 8×1 Zellen



(a) kombinierte, detaillierte Darstellung (Flächen, Kanten, Ecken)



(b) kombinierte, vereinfachte Darstellung

Abb. 4.4: Comb: Gesamtszenario

Um das bisher theoretisch hergeleitete Kommunikationsmodell auch praktisch zu überprüfen, wird in der nächsten Unterunterabschnitt 4.1.1 das Verhalten des Comb-Benchmarks mithilfe eines MPI-Profilers ausgewertet.

Dabei ist zu beachten, dass Collom et al. [5] in ihrer Arbeit nicht explizit angeben, wie viele Flächen, Kanten und Eckpunkte in einem 3D-Halo-Austausch übertragen werden. Sie beschreiben lediglich, dass insgesamt 26 Nachbarprozesse existieren und dass Halo-Zellen dieser drei Kategorien ausgetauscht werden. Die detaillierte Zerlegung in 6 Flächen, 12 Kanten und 8 Eckpunkte ist daher eine Rekonstruktion basierend auf der geometrischen Struktur einer 3D-Domäne.

Validierung mit mpiP

Um diese Frage zu beantworten, wurde der MPI-Profiler *mpiP* [@18] herangezogen, der erfasst, wie oft MPI-Funktionen an welchen Stellen im Code von welcher Rank-ID aufgerufen werden. Zudem erhält man Informationen bezüglich der Datenmenge dieser Funktionen, ohne dass der Programmcode angepasst werden muss.
Der Profiler wurde exemplarisch für einen non-blocking Testlauf eingesetzt. Die Ergebnisse lassen sich aber konzeptionell auch auf die beiden anderen Varianten übertragen. Im dargestellten Bericht tritt deshalb `Isend` als zentrale Funktion auf, anhand der die Ergebnisse abgeleitet werden.
Der Benchmark-Testlauf wurde mit 64 MPI-Prozessen durchgeführt, die mit einer lokalen Gittergröße von $64 \times 64 \times 64$ Zellen kommunizierten. Jede Zelle trägt drei Variablen des Datentyps *Double*. Der MPI-Profiler lief während des gesamten Testlaufs mit und erfasste die Kommunikation für eine Iteration.

Der folgende Ausschnitt stammt aus dem Output von *mpiP*. Die Tabelle 4.2 fasst dabei die Statistiken zu allen gesendeten Nachrichten zusammen, gruppiert nach Aufrufstellen im Code und MPI-Prozess. In dieser Tabelle tragen die Spalten folgende Semantik:

- **Name**: Die aufgerufene MPI-Funktion
- **Site**: Die Codestelle, an der diese Funktion aufgerufen wurde
- **Rank**: Der zugehörige MPI-Prozess
- **Count**: Die Anzahl der Aufrufe dieser Funktion über den Prozess
- **Max**, **Mean**, **Min**: Maximale, durchschnittliche und minimale Nachrichtengröße in Byte
- **Sum**: Die insgesamt gesendete Datenmenge des Prozesses in Byte

Tab. 4.2: Callsite Message Sent statistics (all, sent bytes)

Name	Site	Rank	Count	Max	Mean	Min	Sum
Isend	40	0	26	9.83e+04	2.34e+04	24	6.084e+05
Isend	103	1	26	9.83e+04	2.34e+04	24	6.084e+05
Isend	166	2	26	9.83e+04	2.34e+04	24	6.084e+05
Isend	…	…	…	…	…	…	…
Isend	3945	63	26	9.83e+04	2.34e+04	24	6.084e+05

Zunächst ist zu erkennen, dass sich die Einträge der einzelnen Aufrufstellen nur in den Spalten *Site* und *Rank* unterscheiden, während alle anderen Werte identisch sind. Dies ist zu erwarten, da jeder Prozess die gleiche Anzahl an Sendeoperationen mit identischen Nachrichtengrößen ausführt.
In der Spalte *Count* ist ersichtlich, dass pro Rank 26 `Isend`-Aufrufe ausgeführt werden, was der Summe aus 6 Flächen, 12 Kanten und 8 Eckpunkten entspricht. Die Spalte *Max* entspricht der Größe einer Flächennachricht. Bei 64^3-Gitterzellen pro Prozess umfasst eine Fläche $64 \times 64 = 4096$ Gitterzellen, multipliziert mit 3 Variablen pro Zelle und 8 Bytes pro Variable ergibt sich $4096 \times 3 \times 8 = 98304$ Bytes. Dies entspricht dem *Max*-Eintrag, abgesehen von einer Differenz von 4 Bytes. Dies ist vermutlich auf die Exponentialdarstellung der Tabelle zurückzuführen.
Die *Mean*-Spalte liefert für diese Betrachtung keine weiterführenden Informationen über einzelne Nachrichtentypen, die *Min*-Spalte hingegen schon. Dort tritt mit 24 Bytes die kleinste Nachricht auf, entsprechend einem Eckpunkt (1 Gitterzelle $\times 3$ Variablen $\times 8$ Bytes $= 24$ Bytes).
Abschließend validiert die *Sum*-Spalte die Zusammensetzung der Kommunikation pro Prozess. Die theoretische Berechnung ergibt $6 \times 98304 + 12 \times 1536 + 8 \times 24 = 608448$ Bytes. Der gemessene Wert beträgt $6,084 \times 10^5$ Bytes (608400 Bytes). Die Differenz von 48 Bytes ist auf dieselbe darstellungsbedingte Abweichung zurückzuführen.

4.1.2 Partitioned Implementierung

Für diese Arbeit wurde das öffentliche Comb-Repository auf GitHub (einschließlich der drei verfügbaren Branches) im Hinblick auf eine mögliche Implementierung von partitioned MPI untersucht. Im Quellcode und in der zugehörigen Dokumentation konnte eine Umsetzung der persistent Kommunikation identifiziert werden. Hinweise auf eine Implementierung der partitioned Kommunikation fanden sich dagegen weder im Code noch in der

README-Datei oder anderen Repository-Dokumenten [@3].
In der verfügbaren Version von Comb ist partitioned MPI also noch nicht Bestandteil des Funktionsumfangs, auch wenn in [5] ein Benchmark beschrieben wird, der sowohl um persistent als auch um partitioned Kommunikation erweitert wurde.
Vor diesem Hintergrund wurde es im Rahmen dieser Arbeit zur Aufgabe gemacht, das Kommunikationsmuster der partitioned Kommunikation in den bestehenden Benchmark zu integrieren. Im Folgenden werden daher der systematische Aufbau und die vorgenommenen Modifikationen der einzelnen Komponenten erläutert.

Repository-Struktur

Um einen Überblick über den Quellcode des Comb-Benchmarks zu vermitteln, wird in Abbildung 4.5 die vereinfachte Ordnerstruktur des Comb-Benchmarks dargestellt. Die markierte Datei `comm_pol_mpi_partitioned.hpp` wurde im Zuge dieser Implementierung neu hinzugefügt.

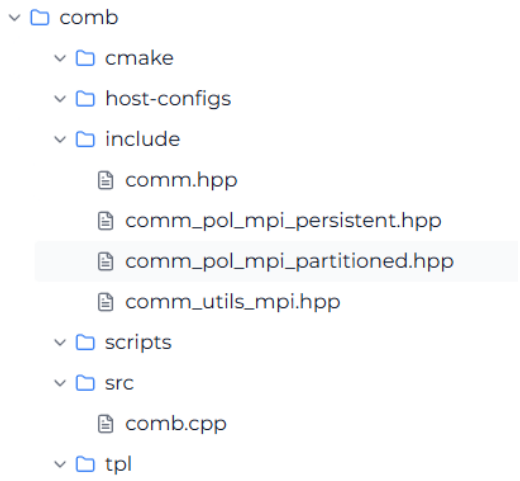


Abb. 4.5: Ordnerstruktur des Comb-Benchmarks

Wrapper-Funktionen

Die Datei `comm_utils_mpi.hpp` im `include`-Verzeichnis dient als Einstiegspunkt der Implementierung. Sie enthält Wrapper-Funktionen aller relevanten non-blocking und persistent MPI-Operationen, die im Benchmark verwendet werden.

Für die partitioned Implementierung wurden folgende Funktionen in dieser Utility-Datei hinzugefügt:

- `MPI_Psend_init`: Initialisierung der partitioned Sendeoperationen
- `MPI_Precv_init`: Initialisierung der partitioned Empfangsoperationen
- `MPI_Pready`: Markierung einer Partition als versandbereit

Weitere notwendige Funktionen wie `Request_free` und `Startall` waren bereits vorhanden, da diese ebenfalls von der persistent Variante verwendet werden.

Persistent MPI-Policy

Die Datei `comm_pol_mpi_persistent.hpp` realisiert die persistent *MPI-Policy* in Comb. Hier werden konkrete Funktionen definiert, die für den Kontrollfluss der Kommunikation genutzt werden. Die Kernfunktionen sind im Wesentlichen:

- `void setup()`: Kommunikationsinitialisierung (jeweils für Sender und Empfänger)
- `void pack()`: Kopieren der Daten aus lokalen Arrays in Sendepuffer
- `void unpack()`: Entpacken in Halo-Bereiche
- `void Isend()`: Sende-Requests ausführen (übernommen)
- `void Irecv()`: Empfangs-Requests ausführen (übernommen)
- `void cleanup()`: Request-Freigabe (übernommen)

Partitioned MPI-Policy

Da das partitioned Kommunikationsmuster auf der persistent Variante basiert, konnte die persistent Policy als Basis übernommen werden. Darauf aufbauend wurden folgende Anpassungen gemacht.

Grundlegendes Zunächst wurden Namensanpassungen im Code durchgeführt, um die Klasse auf partitioned Kommunikation auszurichten. Zusätzlich wurde zu Beginn der neuen Policy eine Variable definiert, die die Anzahl der Partitionen festlegt.

```
int partitions = 2; // beliebig festlegen
```

Diese Variable wird bei der Initialisierung des Benchmarks gesetzt und bestimmt den *Partitionierungsgrad* (Pufferaufteilung) während des Packings, Unpackings sowie des Datentransfers.

Setup In den `setup`-Methoden wurde `Send_init` bzw. `Recv_init` durch `Psend_init` bzw. `Precv_init` ersetzt, was in P. 4.1 kenntlich gemacht wird.

```
// Sender-seitiges Setup
for (int i = 0; i < len; i++) {
  Psend_init(msgs[i]->buf, partitions, ..., requests[i]);
}

// Empfänger-seitiges Setup
for (int i = 0; i < len; i++) {
  Precv_init(msgs[i]->buf, partitions, ..., requests[i]);
}
```

P. 4.1: Vereinfachte Darstellung der partitioned Setup-Schleifen

Der zusätzliche Parameter `partitions` definiert, wie geschildert, den Partitionierungsgrad.

Packing & Unpacking Im Rahmen dieser Implementierung spielt das Packing bzw. Unpacking der auszutauschenden Halo-Daten eine zentrale Rolle. Beim Packing schreibt ein Prozess die benötigten Zellen in separate Nachrichtenpuffer. Beim Unpacking überträgt er die empfangenen Daten aus dem Empfangspuffer in die passenden Halo-Bereiche. Die Umsetzung dieser Funktionen orientiert sich am Codepfad der persistent Policy. Bevor die Kernfunktionen vorgestellt werden, stehen zunächst die grundlegenden Parameter sowie das zentrale Gitterarray der MPI-Policies im Fokus.

Die Policy trägt in ihrer Klassenstruktur:

- `m_variables`: Array der vollständigen Gitterzellen

Die essenziellen Parameter der Funktionen sind:

- `con`: *Execution-Context* für die parallele Ausführung (steuert OpenMP-Threads)
- `msgs[]`: Array mit Halo-Nachrichten-Informationen pro Nachbarprozess
- `len`: Anzahl der Halo-Nachrichten pro Nachbarprozess
- `requests[]`: Vorbereitete Requests für Kommunikation mit Nachbarn

Die hierarchische Struktur von `msgs[]` wird in Abbildung 4.6 demonstriert.

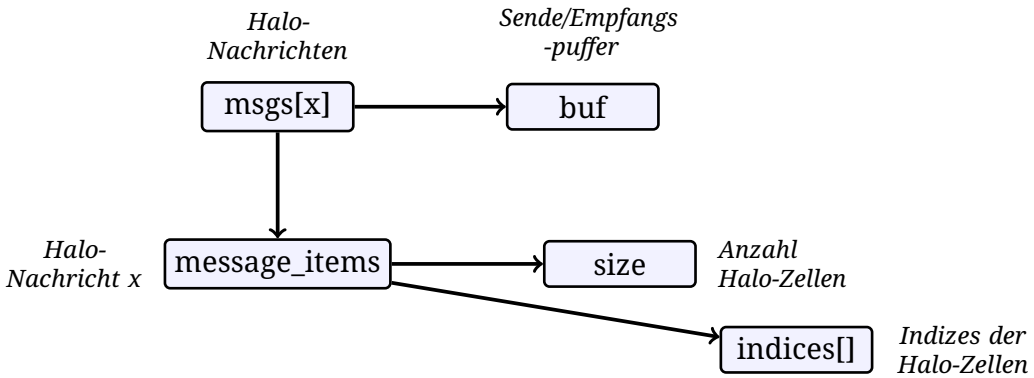


Abb. 4.6: Interne Datenstruktur der Halo-Nachrichten

Der Zugriff auf einen konkreten Halo-Variablenwert erfolgt durch folgende Schritte:

1. `m_variables[var_idx]` wählt das Gitterarray der gewünschten Variable (eine Zelle kann mehrere Variablen haben)
2. `indices[k]` liefert den Gitterindex der k-ten Halo-Zelle
3. `m_variables[var_idx][indices[k]]` ruft den konkreten Variablenwert einer Halo-Zelle ab

Die `indices` mappen beim Packing somit Halo-Zellen aus dem Gitterarray in den zusammenhängenden Puffer und beim Unpacking wieder zurück an die korrekten Halo-Positionen im Gitterarray.

Nun werden die zentralen Funktionen `pack` und `unpack` im Pseudocode (P. 4.2, P. 4.3) angegeben, um deren Funktionsweise kenntlich zu machen.

```
pack(con, msgs[], len, requests[])

    // Parallele Arbeitsgruppe starten
    con.start_group()

    for each message msg with index i in msgs:
        item = msg.message_items
        nitems = item.size
        nvars  = m_variables.size
        items_per_partition = nitems / partitions

        // Parallel über alle Partitionen
        con.for_all(partitions, partition_id):

            // Partition-Bereich bestimmen
            partition_start = partition_id * items_per_partition
            partition_end   = (partition_id == partitions-1)
                            ? nitems
                            : (partition_id+1) * items_per_partition
            part_len    = partition_end - partition_start

            // Buffer-Pointer initialisieren
            buf_current = msg.buf + (partition_start * sizeof(DataT)
            * nvars)

            // Halo-Indizes holen
            indices = item.indices + partition_start

            // Über alle Variablen iterieren
            for var_idx = 0 to nvars-1:
                var_data = m_variables[var_idx]

                // Halo-Zellen in Buffer packen
                for k = 0 to part_len-1:
                    *buf_current = var_data[indices[k]]
                    buf_current++

            // Partition kann senden
            MPI_Pready(partition_id, requests[i])

    // Arbeitsgruppe beenden
    con.finish_group()
```

P. 4.2: Partitioned `pack`: Gitterarray → Puffer + `Pready` pro Partition

```
unpack(con, msgs[], len)

    // Parallele Arbeitsgruppe starten
    con.start_group()

    for each message msg with index i in msgs:

        item  = msg.message_items
        nitems = item.size
        nvars  = m_variables.size
        items_per_partition = nitems / partitions

        // Parallel über alle Partitionen
        con.for_all(partitions, partition_id):

            // Partition-Bereich bestimmen
            partition_start = partition_id * items_per_partition
            partition_end   = (partition_id == partitions-1)
                            ? nitems
                            : (partition_id+1) * items_per_partition
            part_len = partition_end - partition_start

            // Buffer-Pointer initialisieren
            buf_current = msg.buf + (partition_start * sizeof(DataT)
            * nvars)

            // Halo-Indizes holen
            indices  = item.indices + partition_start

            // Über alle Variablen iterieren
            for var_idx = 0 to nvars-1:
                var_data = m_variables[var_idx]

                // Daten dem Empfangspuffer rauspacken
                for k = 0 to part_len-1:
                    var_data[indices[k]] = *buf_current
                    buf_current++

    // Arbeitsgruppe beenden
    con.finish_group()
```

P. 4.3: Partitioned unpack: Puffer → Gitterarray

Ablauf und Reihenfolge der Operationen

Um die Ausführungsreihenfolge der Funktionsaufrufe während der Kommunikation zu bestimmen, wurden die im Paper [5] aufgeführten Algorithmen zur persistent und partitioned Variante analysiert.

Algorithm 1 Persistent Exchange [5]

Input: `msgs_info`, `requests`
{message information (process pairs, data sources), requests storage array}

parallel region {OpenMP Region}
// Copy data from mesh to contiguous message buffer
`pack(msgs_info)`

// Begin communication
`MPI_Startall(requests)`

// Wait on messages
`MPI_Waitall(requests)`

parallel region {OpenMP Region}
// Copy values from message buffer into mesh
`unpack(msgs_info)`

Wie in Algorithmus 1 zu sehen, ist der zeitliche Ablauf bei persistent Kommunikation folgendermaßen strukturiert:

1. Daten werden innerhalb einer OpenMP-Region abhängig von der Anzahl der genutzten Threads pro Prozess gepackt
2. Anschließend wird die Kommunikation gestartet (`Startall`)
3. Alle zu sendenden Nachrichten der aktuellen Iteration werden synchronisiert (`Waitall`)
4. Danach werden die Daten auf Empfängerseite entpackt

Algorithm 2 Partitioned Exchange [5]

Input: `msgs_info`, `requests`
{message information (process pairs, data sources), requests storage array}

// Begin communication
`MPI_Startall(requests)`

parallel region {OpenMP Region}
// Copy data from mesh to contiguous message buffer
`pack(msgs_info)`
// Mark partition of current thread as ready
`MPI_Pready(partition)`

// Wait on messages
`MPI_Waitall(requests)`

parallel region {OpenMP Region}
// Copy values from message buffer into mesh
`unpack(msgs_info)`

In Algorithmus 2 dahingegen unterscheidet sich der Zeitverlauf der partitioned Variante grundlegend:

1. Kommunikation wird vor dem Packing gestartet (`Startall`)
2. Innerhalb der OpenMP-Region wird der Sendepuffer gepackt
3. Sobald eine Partition fertig mit dem Packing ist, wird `Pready` aufgerufen
4. Anschließend wird auf den Erhalt aller Nachrichtenpakete der aktuellen Iteration gewartet (`Waitall`)
5. Danach wird mit dem Unpacking fortgefahren

Diese Ausführungsreihenfolge ist elementar für die Implementierung und wurde folglich in den Benchmark integriert.

In der `comm.hpp`-Datei wird diese Reihenfolge gesteuert. Der folgende Codeabschnitt (P. 4.4) zeigt die ursprüngliche Reihenfolge für die persistent Kommunikation.

```
for (IdxT i = 0; i < num; i++)
{
  m_sends.messages.allocate(...);
  m_sends.messages.pack(..., requests);
  m_sends.messages.wait_pack_complete(...);
```

```
6     m_sends.messages.Isend(..., &requests[i]);
7   }
```

P. 4.4: Persistent Kommunikationsreihenfolge: pack → Isend

Für die partitioned Variante wurde der Code (P. 4.5) mit bedingter Logik integriert.

```
1   for (IdxT i = 0; i < num; i++)
2   {
3     m_sends.messages.allocate(...);
4
5     if (partitioned)
6     {
7       // Partitioned Variante: Isend zuerst
8       m_sends.messages.Isend(..., &requests[i]);
9       m_sends.messages.pack(..., requests);
10      m_sends.messages.wait_pack_complete(...);
11    }
12    else
13    {
14      // Persistent Variante: pack zuerst
15      m_sends.messages.pack(..., requests);
16      m_sends.messages.wait_pack_complete(...);
17      m_sends.messages.Isend(..., &requests[i]);
18    }
19  }
```

P. 4.5: Partitioned Kommunikationsreihenfolge: Isend → pack

Die Puffer werden hier für alle Varianten identisch alloziert. Danach wird geprüft, ob der `partitioned`-Flag auf `true` gesetzt ist. Wenn ja, wird die korrekte Reihenfolge für partitioned MPI erzwungen.

Threading-Modus

Zu diesem Punkt wurde gedacht, dass die Implementierung vollständig und funktionsfähig ist. Dies war zwar in einigen Testläufen der Fall, aber in den meisten Fällen geriet der Benchmark in einen Deadlock, insbesondere wenn die Iterationszyklen über 10 hinausgingen. Im Rahmen der Fehlersuche wurden die Abschnitte zu den Thread-Level in der MPI-Dokumentation [21] herangezogen. Die Datei `comb.cpp` im `src`-Verzeichnis enthielt zu Beginn den Ausschnitt P. 4.6.

```
#ifdef COMB_ENABLE_MPI
  int required = MPI_THREAD_FUNNELED;
  int provided = detail::MPI::Init_thread(&argc, &argv, required);
#endif
```

P. 4.6: MPI-Umgebung initialisieren mit `MPI_THREAD_FUNNELED`

Der Benchmark fordert hier `MPI_THREAD_FUNNELED` an, was bedeutet, dass zwar mehrere Threads innerhalb eines Prozesses agieren können, aber nur ein einziger Thread MPI-Aufrufe ausführen darf.
Die Ursache der Deadlocks liegt darin, dass bei der partitioned Kommunikation mehrere Threads `MPI_Pready` aufrufen, obwohl der angeforderte Threading-Level dieses Verhalten untersagt (vgl. Abschnitt 2.2).

Wie in P. 4.7 gezeigt, besteht die Lösung darin, den Threading-Level auf `MPI_THREAD_MULTIPLE` zu setzen.

```
#ifdef COMB_ENABLE_MPI
  int required = MPI_THREAD_MULTIPLE;
  int provided = detail::MPI::Init_thread(&argc, &argv, required);
#endif
```

P. 4.7: MPI-Umgebung initialisieren mit `MPI_THREAD_MULTIPLE`

Nach dieser Anpassung können die Testläufe ohne Deadlocks abgeschlossen werden, womit die Implementierung der partitioned Variante innerhalb des Comb-Benchmarks abgeschlossen ist.

Zusammenfassung

Die Implementierung von partitioned MPI in Comb erfordert zusammengefasst folgende wesentliche Schritte:

1. Hinzufügen von Wrapper-Funktionen für partitioned MPI in der Datei `comm_utils_mpi.hpp`:
   - `Psend_init`
   - `Precv_init`
   - `Pready`
2. Erstellung einer Policy-Klasse (`comm_pol_mpi_partitioned.hpp`) auf Basis der persistent Policy mit Anpassung der Methoden:

- `void setup`
- `void pack`
- `void unpack`

3. Korrekte Ausführungsreihenfolge der partitioned Operationen festlegen (in `comm.hpp`)
4. Umstellung des Threading-Levels auf `MPI_THREAD_MULTIPLE` zur Unterstützung gleichzeitiger MPI-Aufrufe paralleler Threads

4.2 Comb-Studien: Methodik und experimentelle Konfiguration

In diesem Abschnitt werden zunächst die verwendete Hardware sowie der eingesetzte Software-Stack beschrieben. Des Weiteren wird auf die globale und die prozessinterne Parallelisierungsstrategie eingegangen. Anschließend werden die in den Comb-Studien verwendeten Konfigurationen erläutert, bevor darauf aufbauend die Bewertungsstrategie und die zugrunde liegenden Metriken definiert werden.

4.2.1 Hardwareumgebung und Software-Stack

Die Testläufe wurden auf der "Partition" `ki-parallel` des HPC-Clusters Mogon KI der Johannes Gutenberg-Universität Mainz [@36] durchgeführt, dessen Compute Nodes über InfiniBand verbunden sind. Die Compute Nodes nutzen *AMD EPYC 7713* Prozessoren und verfügen über zwei CPU-Sockel mit jeweils 64 Kernen und einem Hardware-Thread pro Kern. Somit stehen pro Node insgesamt 128 physische Kerne ohne *Simultaneous Multithreading* (SMT) zur Verfügung. Ohne SMT kann jeder physische Kern immer nur einen Thread gleichzeitig ausführen. In der Partition `ki-parallel` sind insgesamt 94 Compute Nodes vorhanden, die mit unterschiedlichem Arbeitsspeicher (RAM) ausgestattet sind: 29 Nodes mit 248 GB und 65 Nodes mit 504 GB. Die hier genannten Hardwareinformationen wurden über Systemabfragen ermittelt. Für die Testläufe kamen jeweils zwischen 1 und 32 Nodes zum Einsatz, wobei die zugewiesenen Nodes dynamisch vom *Slurm-Scheduler* (vgl. Abschnitt 2.1) vergeben werden. Als Kommunikationsschnittstelle kommt die systemseitig

bereitgestellte MPI Implementierung Open MPI 5.0.3 zum Einsatz, während der Programmcode mit der GNU Compiler Collection (GCC) in Version 13.3.0 kompiliert wurde.

4.2.2 Parallelisierungsstrategie und Thread-Mapping

Die Konfiguration der Testläufe orientiert sich an der hybriden MPI+OpenMP-Parallelisierung aus der Referenzstudie [5]. Dort werden hauptsächlich pro Node 32 MPI-Prozesse mit jeweils zwei OpenMP-Threads betrieben, wobei die Threads per SMT auf denselben physischen Kernen ausgeführt werden. Um diese prozessinterne Parallelität auf dem Mogon KI Cluster abzubilden, wurde die Thread-Zuweisung an die fehlende SMT-Unterstützung angepasst: Jedem MPI-Prozess werden zwei dedizierte physische CPU-Kerne zugewiesen, wodurch ebenfalls zwei OpenMP-Threads pro Prozess resultieren. In den Standardkonfigurationen entspricht dies 32 MPI-Prozessen $\times$ 2 Threads = 64 Threads pro Node.
Für die hier betrachteten Läufe wird ein 1:1-Partitions-Mapping verwendet. Die Strong, Weak und Size-Increase Studien verwenden dabei konstant zwei Threads bzw. zwei Partitionen pro MPI-Prozess, während die Threads-pro-Prozess Studie diese Anzahl von zwei bis 64 variiert.

4.2.3 Parameter für Teststudien

Für die in dieser Arbeit durchgeführten Experimente werden die relevanten Comb-Parameter in zwei Gruppen eingeteilt. Dabei handelt es sich um konstante Parameter über alle Studien hinweg und variable Parameter je nach Testlauf. Diese Parameter wurden aus der Referenzarbeit [5] abgeleitet und an die Gegebenheiten des verwendeten Clusters angepasst.

Konstant verwendete Parameter

- `vars`: Legt fest, wie viele Variablenwerte pro Gitterzelle gespeichert werden. Hierbei wird `-vars` 3 verwendet, um pro Zelle drei Variablen abzubilden.
- `ghost`: Steuert die Breite der Halo-Bereiche in jeder Raumrichtung. Durchgängig `-ghost` 1_1_1, passend zum 27-Punkt-Stencil.

- `cycles`: Anzahl der Iterationen des Stencil-Codes. 1000 Zyklen, um Initialisierungs- und Setup-Kosten zu amortisieren und den iterativen Charakter optimierter Varianten abzubilden.
- `periodic`: Periodische Randbedingungen in allen Dimensionen, ohne spezielle Randprozesse (`-periodic` 1_1_1).

Je nach Studie variierte Parameter

- `grid`: Globale Gittergröße in x-, y- und z-Richtung.
- `divide`: Die Prozessgitter-Zerlegung: wird variiert, um verschiedene Prozesszahlen und entsprechende Domänendekomposition zu setzen.

4.2.4 Definition der Teststudien

Inspiriert durch die experimentellen Methoden von Collom et al. [5] werden vier zentrale Teststudien durchgeführt, um die Performance der MPI-Kommunikationsmuster umfassend zu evaluieren.

Strong Scaling

Gemäß dem *Amdahlschen* Modell wird bei konstanter Gesamtproblemgröße das Verhalten unter zunehmendem Ressourceneinsatz untersucht [1, @19]. Konkret wird in dieser Studie dazu das globale Gitter in seiner Größe fixiert und die eingesetzten MPI-Prozesse systematisch erhöht.

Weak Scaling

Im Sinne von *Gustafsons* Skalierungsargumentation wird untersucht, wie sich die Laufzeit verhält, wenn die Arbeitslast pro Prozess konstant bleibt und die globale Problemgröße proportional zur Prozessanzahl wächst [@19, 11]. Entsprechend wird das lokale Gitter pro Prozess konstant gehalten, während sich bei einer Erhöhung der Prozessanzahl das globale Gitter im gleichen Maß vergrößert.

Size-Increase

In dieser Untersuchung wird die Anzahl der MPI-Prozesse fixiert, während die Nachrichtengröße über die Skalierung der Halo-Flächen gezielt variiert wird. Ziel ist es, den Einfluss des steigenden Datenvolumens auf die Kommunikationsleistung zu isolieren.
Statt der Zellenzahl von Collom et al. [5] wird hier die Datenmenge in KiB verwendet. Diese Darstellung bietet einen greifbareren Bezug zum physischen Datentransfer als die rein problembezogene Anzahl an Halo-Zellen.
Die Umrechnung einer einzelnen Referenzfläche in die entsprechende Datengröße erfolgt nach der Formel:

$$\text{Größe [KiB]} = \frac{N_{\text{Zellen(Fläche)}} \times 3 \text{ Vars} \times 8 \text{ Byte (Double)}}{1024} \tag{4.1}$$

Dieser Wert repräsentiert die Datenmenge einer einzelnen zu versendenden Halo-Fläche und dient als Basis für die Analyse bei steigender Last.

Thread-pro-Prozess Studie

Diese Untersuchung analysiert die Balance zwischen MPI-Prozessen und OpenMP-Threads. Hier werden konstant 2048 CPU-Kerne und 32 Nodes in allen Durchläufen eingesetzt. Dabei wird aufgezeigt, wie sich die Aufteilung zwischen MPI-Prozessen und Threads auf die verschiedenen Kommunikationsmuster auswirken kann. Jeder Datenpunkt auf der Threads-pro-Prozess-Achse bedingt dabei eine komplementäre Anzahl an MPI-Prozessen (und damit Prozesse pro Node):

- 64 Threads pro Prozess: Insgesamt 32 MPI-Prozesse (1 Prozess/Node)
- 32 Threads pro Prozess: Insgesamt 64 MPI-Prozesse (2 Prozesse/Node)
- 16 Threads pro Prozess: Insgesamt 128 MPI-Prozesse (4 Prozesse/Node)
- 8 Threads pro Prozess: Insgesamt 256 MPI-Prozesse (8 Prozesse/Node)
- 4 Threads pro Prozess: Insgesamt 512 MPI-Prozesse (16 Prozesse/Node)
- 2 Threads pro Prozess: Insgesamt 1024 MPI-Prozesse (32 Prozesse/Node)

4.2.5 Metriken und Bewertungsstrategie

Parallelisierungsfähigkeit

Der Speedup $S(n)$ beschreibt das Verhältnis der Ausführungszeit einer Konfiguration mit einem Prozess (T_1) zur Ausführungszeit mit n Prozessen (T_n) [8]. Da die Messreihen systematisch ab einer Anzahl von 32 Prozessen durchgeführt wurden, wird diese Konfiguration als Referenzpunkt für die Berechnung des Speedups definiert. Die Laufzeit bei 32 MPI-Prozessen dient somit als Referenzlaufzeit T_{ref}:

$$S(n) = \frac{T_{\text{ref}}}{T_n} \tag{4.2}$$

Die zwei folglich eingeführten Effizienzmetriken orientieren sich an Dematties et al. [7], wobei analog zum Speedup die Referenzlaufzeit T_{ref} statt T_1 verwendet wird:

- **Strong-Scaling-Effizienz:** Da beim Strong Scaling die Problemgröße konstant bleibt und die Prozessanzahl variiert, ist der Speedup allein als Effizienzmaß nicht ausreichend. Daher wird der Skalierungsfaktor $\frac{32}{n}$ einbezogen, um den Speedup auf den Ressourceneinsatz relativ zur Referenzkonfiguration zu normieren:

  $$E_{\text{strong}}(n) = \frac{S(n) \cdot 32}{n} = \frac{T_{\text{ref}}}{T_n} \cdot \frac{32}{n} \tag{4.3}$$

  Ein Wert von $E = 1{,}0$ repräsentiert ideale Skalierung. Werte $E < 1{,}0$ weisen auf negative Skalierungseffekte hin, während $E > 1{,}0$ auf superlinearen Speedup hindeutet [27]. Denn damit übersteigt der Speedup den Skalierungsfaktor der eingesetzten Ressourcen.

- **Weak-Scaling-Effizienz:** Da die globale Problemgröße proportional zur Prozessanzahl wächst, bearbeitet jeder Prozess über alle Messpunkte hinweg eine gleich große Arbeitslast. Daher entspricht die Effizienz in diesem Fall dem Verhältnis der Referenzlaufzeit zur Laufzeit bei n Prozessen:

  $$E_{\text{weak}}(n) = \frac{T_{\text{ref}}}{T_n} \tag{4.4}$$

  Ein stabiler Wert von $E \approx 1{,}0$ bestätigt, dass die Ausführungszeit konstant bleibt, obwohl sowohl Problemgröße als auch Ressourcenanzahl

proportional zueinander wachsen. Dies indiziert, dass der zusätzliche Ressourceneinsatz vollständig zur Bewältigung der zusätzlichen Last genutzt wird.

Diese Effizienzen kommen gezielt im Rahmen der Strong und Weak Scaling Studien zum Einsatz, um die generelle Parallelisierungsfähigkeit der Anwendung zu bewerten. Da die verschiedenen Kommunikationsmuster in diesen Beobachtungen ein qualitativ ähnliches Skalierungsverhalten aufzeigen und sich ihre absoluten Verläufe primär durch einen konstanten Offset unterscheiden, wird für die Auswertung ausschließlich die non-blocking Variante als repräsentative Basis verwendet.

Vergleich der Kommunikationsmuster

Ein wesentlicher Teil der Untersuchung liegt im direkten Vergleich der verschiedenen MPI-Kommunikationsmuster. Angesichts der starken Annäherung der absoluten Laufzeitkurven ist es in der grafischen Darstellung oft schwierig, die spezifischen Performance-Unterschiede klar voneinander abzugrenzen. Um diese Unterschiede der optimierten Muster dennoch präzise zu quantifizieren, wird non-blocking MPI als Baseline definiert. Somit können durch eine relative Gegenüberstellung die Vorteile der spezialisierten Implementierungen isoliert betrachtet werden. Der spezifische Leistungsvorteil gegenüber dem Standard wird dabei als relativer Speedup in Prozent berechnet:

$$S_{rel}[\%] = \left(\frac{T_{std}}{T_{opt}} - 1\right) \times 100 \tag{4.5}$$

Dabei repräsentiert T_{std} die Laufzeit des non-blocking Musters und T_{opt} die der jeweiligen optimierten Variante. Ein positiver Wert verdeutlicht somit den prozentualen Zeitgewinn, den die optimierte Kommunikation gegenüber der Baseline erzielt.

Vergleich mit Collom et al.

Zusätzlich zu den eigenen Messreihen werden die im vorherigen Abschnitt beschriebenen Studien den Ergebnissen der Referenzarbeit von Collom et al. [5] gegenübergestellt, um die beobachteten Trends der Kommunikationsmuster einzuordnen.
Dabei ist zu beachten, dass sich die untersuchten Prozesszahlen zwischen

beiden Arbeiten unterscheiden. Collom et al. variieren ihre Experimente im Bereich von 64 bis 4096 MPI-Prozessen. Diese Arbeit deckt hingegen abhängig von Studie und Konfiguration den Bereich von 32 bis 1024 MPI-Prozessen ab.

4.3 MPI-Ring-Benchmark

Der MPI-Ring-Benchmark orientiert sich an der Size-Increase Studie und untersucht isoliert den Einfluss der Nachrichtengröße innerhalb einer Kommunikation in einem Ringtopologie-Szenario. Der Benchmark implementiert dabei ein eindimensionales Kommunikationsmodell, bei dem jeder MPI-Prozess pro Iteration Daten mit seinem linken und rechten Nachbarn austauscht. Pro Durchlauf werden schließlich zwei Nachrichten gesendet sowie auch empfangen.

4.3.1 Setup und Parameterwahl

Der MPI-Ring-Benchmark ist wie folgt konfiguriert:

- **Topologie**: Eindimensionaler Ring mit 1024 MPI-Prozessen auf 32 Nodes
- **Nachrichtengröße**: Angelehnt an die Size-Increase Studie (Abschnitt 4.2.4) wird die Nachrichtengröße über eine abstrakte Zellanzahl definiert:

  $$\text{Größe einer Nachricht [KiB]} = \frac{\texttt{cells_per_message} \times 3\text{Vars} \times 8\text{Byte}}{1024}$$

  Die Zellanzahl wird systematisch von 256 bis 262144 variiert, entsprechend einem Bereich von 6 KiB bis 6 MiB.
- **Iterationen**: 1000
- **Zeitmessung**: `MPI_Wtime()`, sowie Aggregation mittels `MPI_Reduce` (Mittelwert)
- **Vergleich**: non-blocking Kommunikation (`Isend`/`Irecv`) vs. persistent Kommunikation (`Send_init`/`Recv_init` + `Startall`)

Es ist wichtig anzumerken, dass die kommunizierenden Prozesse keine echten Halo-Bereiche besitzen, die gefüllt werden müssten. Der Benchmark simuliert lediglich den Datenaustausch des Comb-Benchmarks auf konzeptioneller Ebene. Entsprechend werden keine realen Zellwerte ausgetauscht, sondern Dummy-Daten, deren Größe den in der Size-Increase Studie verwendeten Halo-Flächen entspricht.

4.3.2 Initialisierung und Topologie

Nach der Initialisierung von MPI bestimmt jeder Prozess seine beiden Kommunikationspartner im Ring, wie in P. 4.8 gezeigt wird.

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

int left  = (rank - 1 + size) % size;
int right = (rank + 1) % size;
```

P. 4.8: MPI-Ring-Benchmark: Initialisierung und Topologie

Damit ergibt sich eine zyklische Nachbarschaftsbeziehung der Form:

$$\ldots \leftrightarrow 1022 \leftrightarrow 1023 \leftrightarrow 0 \leftrightarrow 1 \leftrightarrow 2 \leftrightarrow \ldots$$

4.3.3 Puffer-Management

Für jede Kommunikationsrichtung werden dedizierte Sende- und Empfangspuffer alloziert (P. 4.9).

```
double *send_left  = malloc(bytes_per_msg);
double *send_right = malloc(bytes_per_msg);
double *recv_left  = malloc(bytes_per_msg);
double *recv_right = malloc(bytes_per_msg);
```

P. 4.9: MPI-Ring-Benchmark: Puffer-Management

Damit verfügt jeder Prozess über vier Puffer, deren Größe linear mit der Zellenanzahl skaliert. Diese Trennung ist notwendig, um den bidirektionalen Datentransfer im Ring korrekt abzubilden.

4.3.4 Non-blocking Kommunikation

P. 4.10 illustriert, dass bei der non-blocking Variante die Initialisierung und Ausführung der Kommunikation innerhalb jeder Iteration miteinander gekoppelt sind.

```
t_start = MPI_Wtime();

MPI_Request reqs[4];
for (int it = 0; it < iters; ++it) {
    MPI_Irecv(recv_left,  doubles_per_msg, MPI_DOUBLE, left,  100,
              MPI_COMM_WORLD, &reqs[0]);
    MPI_Irecv(recv_right, doubles_per_msg, MPI_DOUBLE, right, 101,
              MPI_COMM_WORLD, &reqs[1]);
    MPI_Isend(send_right, doubles_per_msg, MPI_DOUBLE, right, 100,
              MPI_COMM_WORLD, &reqs[2]);
    MPI_Isend(send_left,  doubles_per_msg, MPI_DOUBLE, left,  101,
              MPI_COMM_WORLD, &reqs[3]);

    MPI_Waitall(4, reqs, MPI_STATUS_IGNORE);
}
```

P. 4.10: MPI-Ring-Benchmark: non-blocking Kommunikation

4.3.5 Persistent Kommunikation

In P. 4.11 wird gezeigt, dass dahingegen persistent MPI die Initialisierung und Ausführung der Kommunikation voneinander trennt. Zusätzlich werden die erstellten persistent Requests nach Abschluss der Kommunikationsschleife wieder explizit freigegeben, um die zugehörigen Ressourcen korrekt zu bereinigen.

```
t_start = MPI_Wtime();

MPI_Request reqs[4];

MPI_Send_init(send_right, doubles_per_msg, MPI_DOUBLE, right, 100,
              MPI_COMM_WORLD, &reqs[0]);
MPI_Send_init(send_left,  doubles_per_msg, MPI_DOUBLE, left,  101,
              MPI_COMM_WORLD, &reqs[1]);
MPI_Recv_init(recv_left,  doubles_per_msg, MPI_DOUBLE, left,  100,
              MPI_COMM_WORLD, &reqs[2]);
MPI_Recv_init(recv_right, doubles_per_msg, MPI_DOUBLE, right, 101,
              MPI_COMM_WORLD, &reqs[3]);

```

```
14  for (int it = 0; it < iters; ++it) {
15      MPI_Startall(4, reqs);
16      MPI_Waitall(4, reqs, MPI_STATUS_IGNORE);
17  }
18
19  MPI_Request_free(&reqs[0]);
20  MPI_Request_free(&reqs[1]);
21  MPI_Request_free(&reqs[2]);
22  MPI_Request_free(&reqs[3]);
```

P. 4.11: MPI-Ring-Benchmark: persistent Kommunikation

4.3.6 Messmethodik

Die Zeitmessung erfolgt für beide Kommunikationsvarianten identisch. Nach einer initialen Synchronisation aller Prozesse mittels `MPI_Barrier` wird pro Prozess die Gesamtdauer von 1000 Iterationen erfasst, analog zur Methodik des Comb-Benchmarks. Die Aggregation der gemessenen Zeiten erfolgt über `MPI_Reduce` auf Rank 0 und ergibt nach Division durch die Prozessanzahl den Mittelwert der Kommunikationszeiten über alle Prozesse. Für jede Nachrichtengröße werden fünf unabhängige Durchläufe ausgeführt, deren Laufzeiten anschließend gemittelt werden, um systembedingte Schwankungen zu reduzieren.

4.3.7 Einschränkung: Partitioned Kommunikation

Eine eigenständige Implementierung von partitioned MPI konnte nicht erfolgreich in den MPI-Ring-Benchmark integriert werden. Ursache ist ein bekannter Fehler in Open MPI 5.0.3 [@32], bei dem mehrere `MPI_Psend_init`-Aufrufe desselben Prozesses zu unterschiedlichen Zielen zu Deadlocks führen. Da die entsprechende Lösung [@25] auf dem verwendeten HPC-System nicht eigenständig eingebunden werden konnte, beschränkt sich der Benchmark auf den Vergleich zwischen non-blocking und persistent Kommunikation. Da partitioned Kommunikation grundlegend auf persistent Requests aufbaut, lassen sich die im Benchmark beobachteten Effekte der persistent Variante zumindest teilweise als Indikator für das erwartete Verhalten der partitioned Variante interpretieren.

Anmerkung: Im Comb-Benchmark funktioniert die partitioned Kommunikation mit Open MPI 5.0.3 hingegen ohne Probleme, obwohl dort ebenfalls mehrere `MPI_Psend_init`-Aufrufe zu unterschiedlichen Nachbarn durchgeführt werden. Die genaue Ursache für diesen Unterschied bleibt unklar.

4.4 RDTSC-Benchmark

`RDTSC` (Read Time-Stamp Counter) ist eine x86-Prozessorinstruktion. Sie wird zum Auslesen des CPU-internen *Time-Stamp Counters* (TSC) genutzt, der die Anzahl der seit dem letzten CPU-Reset vergangenen Taktzyklen repräsentiert [@4, 15]. Auf dieser Grundlage lässt sich ein Micro-Benchmark entwerfen, der ausgewählte Schritte des Kommunikationsverhaltens in Taktzyklen gegenüberstellt.

4.4.1 Point-to-Point Management Layer

Wie bereits in Abschnitt 2.4 erläutert, stellt UCX eine universelle Kommunikationsschicht dar, die die Verbindung zwischen MPI und der zugrunde liegenden Hardware realisiert. Die konkrete Einbindung von UCX erfolgt jedoch implementierungsspezifisch.
Open MPI nutzt den *Point-to-Point Management Layer* (PML), um Punkt-zu-Punkt Kommunikation zwischen MPI-Prozessen zu realisieren. Typische MPI-Aufrufe wie `Send`, `Recv` sowie deren non-blocking Varianten `Isend` und `Irecv` werden intern auf PML-Funktionen abgebildet.
Im Fall des PML-UCX-Moduls werden diese Aufrufe an UCX weitergereicht, welches den Datentransfer über die verfügbare Hardware durchführt. Der PML-Layer dient dabei als Vermittler, der die Semantik der MPI Kommunikation auf UCX-Operationen abbildet. Diese Abfolge wird in Abbildung 4.7 dargestellt.

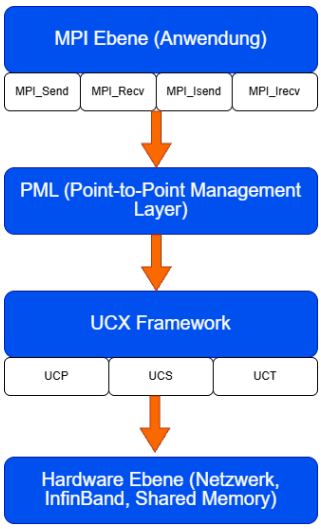


Abb. 4.7: PML-Layer als Vermittlungsschicht (angelehnt an [31])

Kontrollfluss

Im Folgenden werden exemplarisch die Abläufe eines non-blocking sowie eines persistent Sendeaufrufs im PML-UCX-Modul [@23] genauer erläutert. Die Beschreibung orientiert sich dabei am Quellcode dieses Moduls.

Non-blocking Fall Beim Aufruf von `Isend` werden die Parameter `buf`, `count`, `datatype`, `dst`, `tag`, `comm` und `request` an den PML-Layer übergeben. Dieser bildet den Aufruf auf `mca_pml_ucx_isend` ab.
Innerhalb dieser Funktion wird der UCX-Endpoint (`ep`), sowie der UCX-interne Nachrichtentag (`ucx_tag`) bestimmt. Anschließend erfolgt die Ausführung von `mca_pml_ucx_common_send`, wobei ein neuer UCX-Request erzeugt und mit dem MPI-Request verknüpft wird. Dieser gesamte Ablauf, einschließlich der Ermittlung von `ep` und `ucx_tag`, wird bei jedem `MPI_Isend`-Aufruf und damit in jeder Iteration vollständig durchgeführt.

Persistent Fall Im persistent Fall erfolgt die Initialisierung über `Isend_init`. Dabei werden `ep` und `ucx_tag` einmalig erzeugt und gemeinsam mit den Argumenten des MPI-Aufrufs in einem internen Request-Objekt gespeichert. Bei jedem nachfolgenden `Start` greift `mca_pml_ucx_start` auf dieses vorbereitete Request-Objekt zu und ruft `mca_pml_ucx_common_send` auf. Die wiederholte Erstellung von Endpoints und Tags entfällt vollständig, wodurch der Initialisierungs-Overhead reduziert werden kann.

4.4.2 Realisierung des RDTSC-Benchmarks

Um den Overhead-Unterschied zwischen non-blocking und persistent MPI Kommunikation isoliert zu bewerten, wurde ein Micro-Benchmark erstellt. Dieser modelliert den Kontrollfluss des PML-UCX-Moduls in Open MPI 5.0.3 [@23] und misst ausschließlich die benötigten Taktzyklen der Senderseite.

Non-blocking Fall (`f_isend_like`)

```
void f_isend_like(int buf, int count, int datatype,
                  int dest, int tag, int comm,
                  simple_request_t *request)
{
    int ep;

    ep = comm + dest;               // mca_pml_ucx_get_ep() like
    int ucx_tag = tag + comm;       // PML_UCX_MAKE_SEND_TAG() like

    // mca_pml_ucx_common_send() like
    int req_update = ep + buf + count +
                     datatype + ucx_tag + comm;
    request->buf = req_update;
}
```

P. 4.12: RDTSC: f_isend_like

Die Funktion in P. 4.12 simuliert `mca_pml_ucx_isend`. In jeder Iteration werden Endpoint und UCX-Tag neu berechnet und anschließend ein repräsentativer Sendeaufruf ausgeführt. Funktionsaufrufe werden hierbei mit Additionen ersetzt. Falls mehrere Parameter involviert sind, sind dementsprechend auch mehrere Additionen einem Funktionsaufruf zugeordnet.

Persistent Fall (f_isend_init_like, f_start_like)

```
void f_isend_init_like(int buf, int count, int datatype,
                       int dest, int tag, int comm,
                       simple_request_t *request)
{
    int ep;
    ep = comm + dest;          // mca_pml_ucx_get_ep() like

    request->buf      = buf;
    request->count    = count;
```

```
    request->datatype = datatype;
    request->ep       = ep;
    request->ucx_tag  = tag + comm; // PML_UCX_MAKE_SEND_TAG() like
    request->comm     = comm;
}
```

P. 4.13: RDTSC: f_isend_init_like

P. 4.13 modelliert `mca_pml_ucx_isend_init`. Endpoint und Tag werden dabei einmalig berechnet und zusammen mit den übrigen Parametern im Request-Objekt abgelegt.

```
void f_start_like(simple_request_t *request)
{
    // mca_pml_ucx_common_send() like
    int req_update = request->ep +
                     request->buf +
                     request->count +
                     request->datatype +
                     request->ucx_tag +
                     request->comm;
    request->buf = req_update;
}
```

P. 4.14: RDTSC: f_start_like

P. 4.14 repräsentiert den Sendevorgang über ein wiederverwendbares Request-Objekt, wie es bei `MPI_Start` der Fall ist. Endpoint und Tag werden dabei nicht erneut bestimmt, sondern aus dem Request entnommen.

Messung über Inline-Assembly

```
// Taktzyklen auslesen (über Inline-Assembly)
static inline uint64_t rdtsc(void) {
    unsigned int lo, hi;
    __asm__ volatile ("rdtsc" : "=a"(lo), "=d"(hi));
    return ((uint64_t)hi << 32) | lo;
}

static const int N = 1000;

uint64_t bench_isend() {
    simple_request_t req = {1, 2, 3, 4, 5, 6};
    uint64_t start = rdtsc();
    for (int i = 0; i < N; ++i) {
```

```
14          f_isend_like(1, 2, 3, 4, 5, 6, &req);
15      }
16      uint64_t end = rdtsc();
17      return (end - start) / N;
18  }
19
20  uint64_t bench_start() {
21      simple_request_t req;
22      uint64_t start = rdtsc();
23      f_isend_init_like(1, 2, 3, 4, 5, 6, &req);
24      for (int i = 0; i < N; ++i) {
25          f_start_like(&req);
26      }
27      uint64_t end = rdtsc();
28      return (end - start) / N;
29  }
```

P. 4.15: RDTSC: Messung über Inline-Assembly

P. 4.15 zeigt die Messlogik beider Varianten. Zunächst wird die Funktion `rdtsc()` mithilfe von *Inline-Assembly* definiert, mit der sich der aktuelle Zählerstand der CPU-Taktzyklen auslesen lässt. Inline-Assembly bedeutet, dass Assemblercode direkt in C-Code eingebunden wird, sodass an dieser Stelle die entsprechende Prozessorinstruktion `rdtsc` ausgeführt wird.
Nach $N = 1000$ Iterationen ergibt die Differenz aus End- und Startwert die insgesamt benötigten Taktzyklen für den gemessenen Codeabschnitt. Anschließend wird dieser Wert durch N dividiert, um den Mittelwert an CPU-Taktzyklen pro Sendeoperation zu erhalten.
Während `bench_isend` den non-blocking Pfad inklusive der wiederholten Bestimmung von Endpoint und UCX-Tag in jeder Iteration erfasst, misst `bench_start` nach einmaliger Initialisierung ausschließlich die wiederholte Ausführung von `f_start_like` auf Basis des vorbereiteten Request-Objekts.

Kompilierung des Benchmarks

Der RDTSC-Benchmark wurde mit GCC in Version 13.3.0 unter Verwendung des C11-Standards kompiliert. Um sicherzustellen, dass keine Optimierungen den Benchmark verfälschen, wurde der Compiler auf Optimierungsgrad `-O0` gesetzt. Dies verhindert weitgehend, dass Funktionsaufrufe oder Berechnungen entfernt oder zusammengefasst werden. Die Kompilierung ist dabei in P. 4.16 dargestellt.

```
gcc -O0 -std=c11 -o rdtsc_bench rdtsc_bench.c
```

P. 4.16: RDTSC: Kompilierung des Benchmarks

Auf diese Weise bleiben die Funktionsaufrufe und Berechnungen im Quellcode erhalten, was für eine verlässliche Messung der CPU-Taktzyklen pro Funktionsaufruf erforderlich ist.

Limitationen und Zweck

Funktionsaufrufe sind nicht äquivalent zu arithmetischen Operationen und tragen andere Kosten mit sich. In dieser Abstraktion werden mehrparametrische Funktionen durch entsprechend viele Additionen modelliert, was die tatsächliche Funktionsweise von Funktionsaufrufen vereinfacht darstellt. Dieser Benchmark erhebt daher keinen Anspruch auf exakte Bestimmung des Overheads-Unterschieds, sondern abstrahiert dessen grundlegende Logik. Dadurch wird der konzeptuelle Unterschied zwischen non-blocking und persistent MPI nachvollziehbar gemacht.

5 Analyse und Auswertung

5.1 Comb-Benchmark

In diesem Abschnitt werden die Ergebnisse der Comb-Studien vorgestellt. Die Strong und Weak Scaling Studien analysieren zusätzlich die Parallelisierungsfähigkeit des Benchmarks unter gegebenen Konfigurationen, während die beiden nachfolgenden Studien ausschließlich den relativen Vergleich der Kommunikationsmuster betrachten. Abschließend werden alle vier Studien mit der Referenzarbeit [5] verglichen.

5.1.1 Strong Scaling Studie

In dieser Versuchsreihe wird das globale Problem auf eine feste Gittergröße von $2048 \times 2048 \times 1024$ Zellen fixiert. Die Anzahl der MPI-Prozesse wird sukzessive von 32 auf 1024 erhöht. Nach der Thread-Konfiguration kommen pro MPI-Prozess zwei Threads (entspricht zwei CPU-Kernen) zum Einsatz. Entsprechend der Methodik des Strong Scalings verringert sich die lokale Arbeitslast pro Prozess mit zunehmendem Ressourceneinsatz proportional.

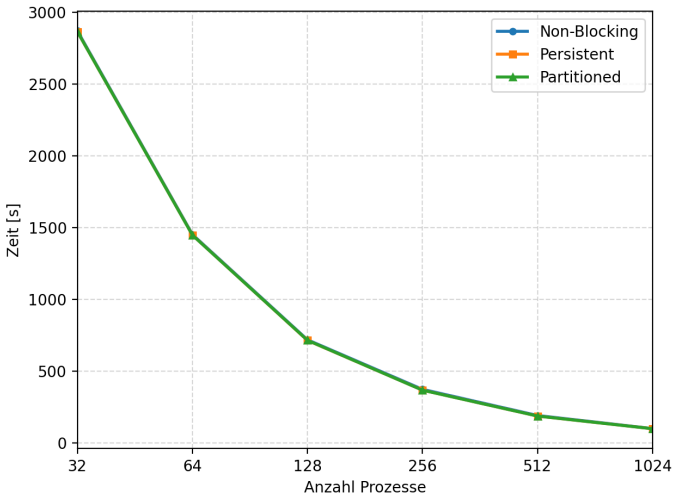


Abb. 5.1: Strong Scaling: Verlauf der absoluten Kommunikationszeiten

56

Tab. 5.1: Strong Scaling: Kommunikationszeiten

Prozesse	Non-blocking [s]	Persistent [s]	Partitioned [s]
32	2870,284	2864,657	2864,673
64	1451,103	1447,239	1446,633
128	717,394	713,119	713,763
256	372,042	366,711	366,186
512	189,802	186,122	186,023
1024	98,696	97,810	97,554

Analyse der Parallelisierungsfähigkeit

Tab. 5.2: Strong Scaling: Effizienz (Referenz: non-blocking MPI)

Prozesse	Skalierungsfaktor	Zeit [s]	Effizienz (E_{strong})
32	1,0	2870,284	1,0000
64	2,0	1451,103	0,9890
128	4,0	717,394	1,0002
256	8,0	372,042	0,9644
512	16,0	189,802	0,9452
1024	32,0	98,696	0,9088

Die Ergebnisse (siehe Tabelle 5.1 und Abbildung 5.1) belegen insgesamt eine gute Skalierbarkeit, wobei sich die Laufzeit bei jeder Verdopplung der Prozesszahl nahezu halbiert. Am größten Messpunkt ist jedoch ein leichter Effizienzabfall sichtbar, der auf zunehmende Skalierungsverluste bei hoher Parallelität hinweist.

Dieses Verhalten wird durch die Effizienzmetrik (Tabelle 5.2) quantitativ bestätigt. Bis zu 512 MPI-Prozessen bleibt die Strong Scaling Effizienz durchgehend über *0,94* und deutet damit auf ein insgesamt konsistentes Parallelisierungsverhalten des Benchmarks hin. Hervorzuheben ist der Messwert bei 128 Prozessen mit einer Effizienz von *1,0002*. Diese leicht *superlineare* Performance-Verbesserung gegenüber dem Referenzwert (T_{ref}) ist aber vermutlich auf Systemrauschen zurückzuführen. Wiederholte Testläufe wären hier notwendig, um diese Vermutung weiter zu prüfen, liegen jedoch nicht mehr im zeitlichen Rahmen dieser Arbeit.
Dieser Effekt tritt jedoch nur in einem begrenzten Bereich auf. Beim Übergang auf 1024 MPI-Prozesse fällt die Effizienz auf ca. *0,909*, und bestätigt damit das Bild zunehmender Skalierungsverluste bei steigender Parallelisierung. Dieser Rückgang ist konsistent mit dem Gesetz von Amdahl [1], da bei fixierter globaler Problemgröße die lokale Arbeitslast pro Prozess abnimmt, während der relative Anteil von Kommunikations-Overhead und Synchronisations-

zeiten zunimmt. Damit deutet dieser Messpunkt darauf hin, dass sich der Benchmark der Skalierungsgrenze im Sinne von Amdahl annähert.

Relativer Performance-Vergleich der Kommunikationsmuster

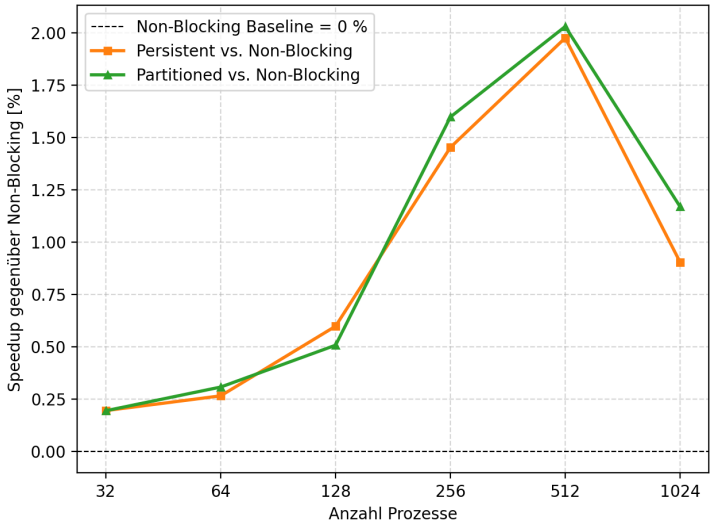


Abb. 5.2: Strong Scaling: Verlauf der relativen Speedups gegenüber non-blocking MPI

Tab. 5.3: Strong Scaling: Relative Speedups gegenüber non-blocking MPI

Prozesse	Persistent [%]	Partitioned [%]
32	0,196	0,196
64	0,267	0,309
128	0,600	0,509
256	1,454	1,599
512	1,977	2,031
1024	0,906	1,171

Im direkten Vergleich der optimierten Kommunikationsmuster gegenüber der non-blocking Referenz (siehe Tabelle 5.3 und Abb. 5.2) zeigen sich Vorteile im Bereich von etwa 0,2 % bis 2 %. Hier weisen die Verläufe eine hohe Ähnlichkeit auf.

Konsistenz zwischen persistent und partitioned MPI Zu beobachten ist, dass die persistent und partitioned Kommunikation in ihrer Performance nahezu gleichauf liegen. Da partitioned auf persistent MPI basiert, lässt sich vermuten, dass die Trennung von Initialisierung und Ausführung den primären Vorteil gegenüber dem non-blocking Verfahren liefert. Ein zusätzlicher Nutzen des

partitioned Datentransfers wird gegenüber der persistent Variante in den Messreihen nicht deutlich sichtbar.

Amortisierung von Kommunikations-Overhead Während der relative Speedup bei 32 Prozessen marginal bleibt, steigt er bei 512 Prozessen auf ca. 2%. Dies könnte daran liegen, dass bei einem kleiner werdenden lokalen Gitter der relative Anteil der Kommunikationsinitialisierung an der Gesamtlaufzeit wächst. Da die persistent und partitioned Varianten diese Initialisierung vor die Iterationen auslagern, entfällt der entsprechende Overhead innerhalb der Hauptschleife im Vergleich zum non-blocking Referenzmuster.

Sättigungseffekte Der relative Speedup zeigt ein Maximum bei 512 Prozessen, sinkt aber bei 1024 Prozessen auf 1%. Die genauen Ursachen für den Rückgang bei 1024 Prozessen lassen sich ohne detailliertes MPI-Profiling nicht eindeutig bestimmen und könnten erneut auf Systemrauschen zurückzuführen sein.

5.1.2 Weak Scaling Studie

In dieser Messreihe wird die Anzahl der MPI-Prozesse von 32 bis 1024 skaliert. Um die Bedingungen für ein Weak Scaling zu erfüllen, bleibt die lokale Gittergröße pro Rank auf $512 \times 512 \times 512$ Zellen fixiert, wodurch die globale Problemgröße proportional zu den eingesetzten Ressourcen ansteigt.

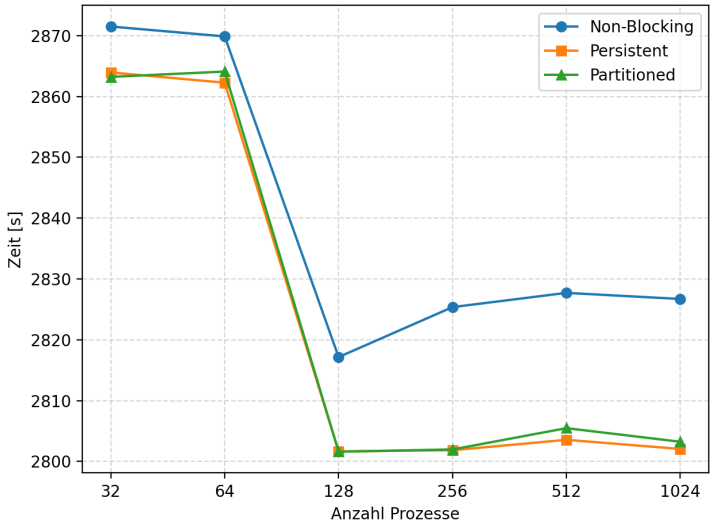


Abb. 5.3: Weak Scaling: Verlauf der absoluten Kommunikationszeiten

Tab. 5.4: Weak Scaling: Kommunikationszeiten

Prozesse	Non-blocking [s]	Persistent [s]	Partitioned [s]
32	2871,504	2863,952	2863,222
64	2869,875	2862,263	2864,100
128	2817,149	2801,645	2801,589
256	2825,366	2801,836	2801,963
512	2827,706	2803,545	2805,464
1024	2826,703	2802,043	2803,227

Analyse der Parallelisierungsfähigkeit

Tab. 5.5: Weak Scaling: Effizienz (Referenz: non-blocking MPI)

Prozesse	Skalierungsfaktor	Zeit [s]	Effizienz (E_{weak})
32	1,0	2871,504	1,0000
64	2,0	2869,875	1,0006
128	4,0	2817,149	1,0193
256	8,0	2825,366	1,0163
512	16,0	2827,706	1,0155
1024	32,0	2826,703	1,0158

Die Ergebnisse (siehe Tabelle 5.4 und Abbildung 5.3) zeigen, dass der Benchmark im gesamten untersuchten Bereich sehr gut skaliert, da die Laufzeit über alle Prozesszahlen hinweg unter der Referenzlaufzeit liegt.

Dies untermauern auch die Ergebnisse aus Tabelle 5.5. Die Weak Scaling Effizienz liegt ab 64 Prozessen durchgängig über *1,0* (maximal *1,0193* bei 128 Prozessen), was auf einen superlinearen Speedup gegenüber dem Referenzwert hindeutet. Dieses Verhalten könnte sich durch Cache-Effekte oder hardwarebedingte Einflüsse bei höheren Prozesszahlen erklären [27]. Eine belastbare Ursachenanalyse würde jedoch zusätzliches Profiling erfordern. Insgesamt sind die Weak Scaling Ergebnisse konform mit Gustafsons Skalierungsargumentation (Effizienz nahe *1,0* bei proportional wachsender Problemgröße) und bestätigen eine solide Parallelisierungsfähigkeit für große Gitterstrukturen.

Relativer Performance-Vergleich der Kommunikationsmuster

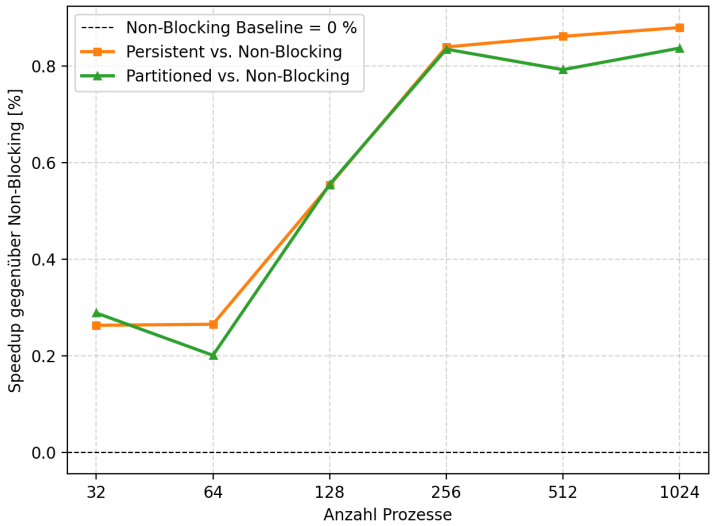


Abb. 5.4: Weak Scaling: Verlauf der relativen Speedups gegenüber non-blocking MPI

Tab. 5.6: Weak Scaling: Relative Speedups gegenüber non-blocking MPI

Prozesse	Persistent [%]	Partitioned [%]
32	0,264	0,289
64	0,266	0,202
128	0,553	0,555
256	0,840	0,835
512	0,862	0,793
1024	0,880	0,837

Der direkte Vergleich der optimierten Kommunikationsmuster gegenüber der non-blocking Referenz (siehe Tabelle 5.6 und Abb. 5.4) zeigt, dass die Performance-Vorteile insgesamt sehr gering ausfallen. Über den gesamten Skalierungsbereich liegen die Verbesserungen unterhalb der 1 %-Marke, was auf eine bereits effiziente non-blocking Implementierung hindeutet. Dennoch lässt sich eine geringfügige, aber konsistente Tendenz zugunsten modernerer Kommunikationsmuster beobachten.

Ausschöpfung des Optimierungspotenzials Erneut zeigen persistent und partitioned nahezu identische Speedups. Da partitioned MPI jedoch keinen signifikanten Mehrwert gegenüber der persistent Variante erzielt, deutet dies darauf hin, dass die spezifischen Vorteile des partitioned Datentransfers bei der gewählten konstanten lokalen Problemgröße begrenzt sind. Diese werden offenbar durch die einmalige Initialisierung weitgehend ausgeschöpft.

Geringfügiger Trend Der relative Speedup nimmt mit steigender Prozesszahl tendenziell zu und erreicht bei 1024 Prozessen mit 0,88 % (persistent) sein Maximum. Auch wenn dieser Zuwachs gering ist, deutet er darauf hin, dass persistent und partitioned MPI den Kommunikations-Overhead bei sehr hohen Prozesszahlen etwas effektiver reduzieren können als das Standardverfahren.

5.1.3 Size-Increase Studie

In dieser Untersuchung wird die Auswirkung der Nachrichtengröße auf die Effektivität der Kommunikationsmuster analysiert. Während die Anzahl der eingesetzten Ressourcen konstant auf 1024 MPI-Prozesse fixiert bleibt, wird die zu übertragende Datenmenge pro Halo-Fläche sukzessive erhöht.

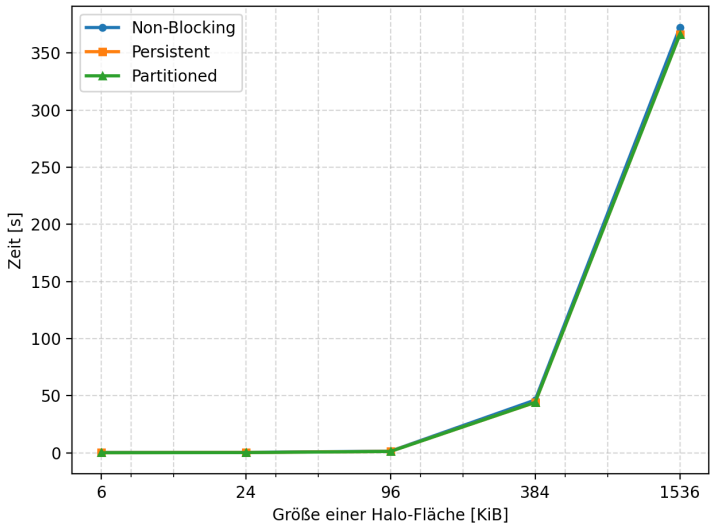


Abb. 5.5: Size-Increase: Verlauf der absoluten Kommunikationszeiten

Tab. 5.7: Size-Increase: Kommunikationszeiten

Halo-Größe [KiB]	Non-blocking [s]	Persistent [s]	Partitioned [s]
6	0,313	0,182	0,137
24	0,454	0,327	0,276
96	1,466	1,302	1,240
384	46,249	44,165	44,199
1536	372,235	366,255	366,083

Relativer Performance-Vergleich der Kommunikationsmuster

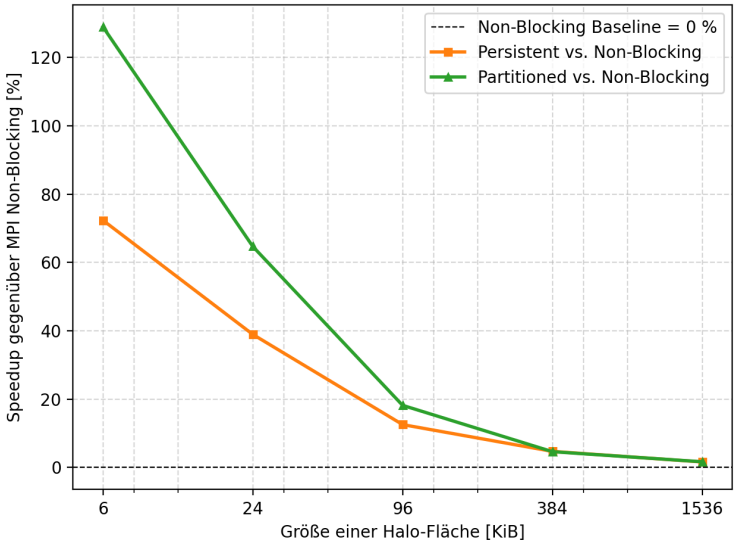


Abb. 5.6: Size-Increase: Verlauf der relativen Speedups gegenüber non-blocking MPI

Tab. 5.8: Size-Increase: Relative Speedups gegenüber non-blocking MPI

Halo-Größe [KiB]	Persistent [%]	Partitioned [%]
6	72,356	128,962
24	38,962	64,705
96	12,565	18,200
384	4,720	4,638
1536	1,633	1,681

Der relative Vergleich gegenüber der non-blocking Referenz (siehe Tabelle 5.8 und Abb. 5.6) zeigt erhebliche Performance-Unterschiede, die stark von der gewählten Nachrichtengröße abhängen.

Dominanz bei kleinen Nachrichten Besonders bei sehr kleinen Halo-Flächen (6 KiB bis 24 KiB) erzielen die optimierten Muster ihre deutlichsten Performance-Vorteile. Partitioned MPI erreicht bei 6 KiB einen Speedup von ca. 129 %, während persistent MPI bei ca. 72 % liegt. Dies verdeutlicht, dass die Amortisierung der Kommunikationsinitialisierung und der partitioned Datentransfer den größten Einfluss haben, solange die reine Netzwerktransferzeit im Verhältnis zum Initialisierungs-Overhead gering bleibt.

Vorteil von partitioned MPI Im Bereich bis 96 KiB zeigt partitioned MPI einen klaren Vorsprung gegenüber der Variante persistent. Hier scheint die Möglichkeit, den Datentransfer Thread-seitig parallel auszuführen, einen zusätzlichen

Performance-Vorteil zu generieren, der über die reine Einsparung durch persistent Requests hinausgeht.

Angleichung bei großen Nachrichten Mit zunehmender Nachrichtengröße (ab 384 KiB) sinkt der relative Vorteil beider optimierten Varianten drastisch ab und stabilisiert sich bei ca. 1,6 % für 1536 KiB. Bei diesen Datenmengen nimmt die Netzwerktransferzeit einen so großen Anteil an der Gesamtlaufzeit ein, dass die softwareseitigen Optimierungen noch kaum ins Gewicht fallen. Entgegen der Erwartung, dass bei größeren Datenmengen die parallele bzw. partitioned Übertragung stärkere Performance-Vorteile zeigt, bestätigt sich dies in den Messreihen nicht. Die Ursachen konnten im Rahmen dieser Arbeit nicht abschließend untersucht werden und könnten unter anderem in implementierungs- oder systembedingten Effekten begründet sein.

5.1.4 Threads-pro-Prozess Studie

Diese Variationsstudie hält sowohl die globale Problemgröße als auch die Gesamtzahl der eingesetzten Rechenressourcen konstant. Die globale Gittergröße ist auf $4096 \times 4096 \times 2048$ Zellen fixiert, während insgesamt 2048 CPU-Kerne (bzw. Threads) und 32 Nodes genutzt werden. Der erste Messpunkt nutzt 32 MPI-Prozesse mit jeweils 64 Threads und der letzte Messpunkt 1024 Prozesse mit jeweils 2 Threads.

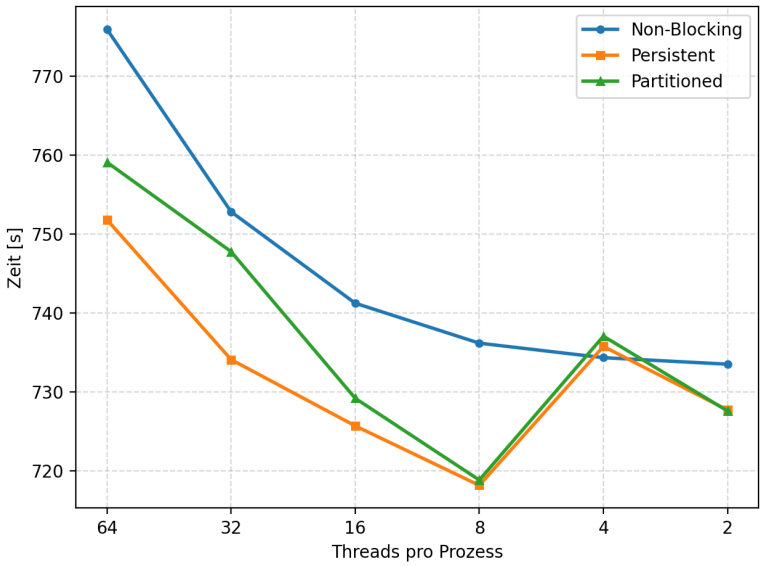


Abb. 5.7: Threads-pro-Prozess: Verlauf der absoluten Kommunikationszeiten

Tab. 5.9: Threads-pro-Prozess: Kommunikationszeiten

Threads/Prozess	Non-blocking [s]	Persistent [s]	Partitioned [s]
64	775,977	751,815	759,114
32	752,831	734,086	747,774
16	741,237	725,678	729,179
8	736,168	718,139	718,806
4	734,329	735,756	737,053
2	733,503	727,687	727,573

Relativer Performance-Vergleich der Kommunikationsmuster

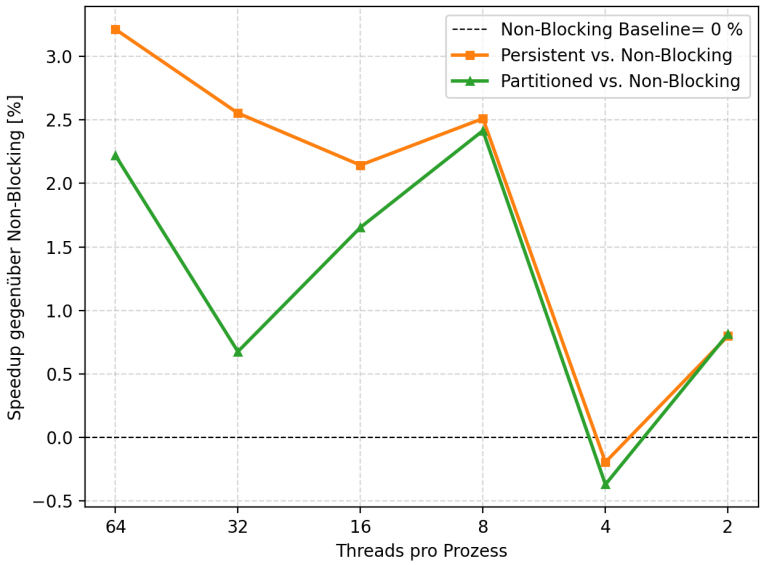


Abb. 5.8: Threads-pro-Prozess: Verlauf der relativen Speedups gegenüber non-blocking MPI

Tab. 5.10: Threads-pro-Prozess: Relative Speedups gegenüber non-blocking MPI

Threads/Prozess	Persistent [%]	Partitioned [%]
64	3,214	2,221
32	2,553	0,676
16	2,144	1,654
8	2,511	2,415
4	-0,194	-0,370
2	0,799	0,815

Der relative Vergleich gegenüber der non-blocking Referenz (siehe Tabelle 5.10 und Abb. 5.8) offenbart durchgehend positive Vorteile der optimierten Muster, mit einer Ausnahme in dieser Studie.

Vorteile bei hoher Thread-Dichte Bei der Konfiguration mit 64 Threads pro Prozess erreichen die optimierten Varianten ihre höchsten relativen Gewinne mit ca. 3,21 % (persistent) und 2,22 % (partitioned). Dies zeigt, dass die Trennung von Kommunikationsvorbereitung und eigentlichem Datentransfer besonders dann einen Vorteil bietet, wenn sehr viele Threads innerhalb eines einzelnen Prozesses arbeiten.

Anomalie bei 4 Threads Anzumerken ist ein "Ausreißer" bei 4 Threads pro Prozess, wo beide optimierten Varianten einen geringfügig negativen Speedup zeigen (ca. -0,2 % bis -0,4 %). Eine fundierte Interpretation würde Messwiederholungen und Fehlerquantifizierung erfordern.

Vergleich persistent vs. partitioned Obwohl partitioned MPI speziell für multithreaded Szenarien entwickelt wurde, liegt es in dieser Studie meist knapp hinter oder gleichauf mit persistent MPI. Dies legt nahe, dass der Hauptanteil der Performance-Vorteile bereits aus der persistent Initialisierung resultiert und der zusätzliche Overhead der Partitionierung in diesem Setup den Zusatznutzen teilweise reduziert.

5.1.5 Diskussion der Ergebnisse im Kontext der Referenzarbeit

Die in dieser Arbeit beobachteten Trends werden nun den Ergebnissen von Collom et al. [5] gegenübergestellt. Die Autoren berichten je nach Konfiguration von deutlichen Speedups gegenüber der Baseline (non-blocking). Die Übertragbarkeit der absoluten Zahlen ist dabei etwas eingeschränkt, da Plattform und MPI Implementierung abweichen. Die Messungen der Referenzarbeit [5] wurden dabei auf dem Quartz-Supercomputer mit MVAPICH2 2.3.7 durchgeführt.

Strong Scaling

Im Strong Scaling liegen die Kommunikationsvarianten in den hier betrachteten Messungen eng beieinander. Gegenüber der non-blocking Referenz werden für persistent und partitioned lediglich geringe Speedups im Bereich von etwa 0,2-2 % beobachtet. Persistent und partitioned unterscheiden sich dabei kaum.
Collom et al. [5] berichten im Strong Scaling deutlich ausgeprägtere Effekte. Für persistent wird ein maximaler Speedup von bis zu 37 % gegenüber der

Baseline angegeben (bei 2048 Prozessen), während partitioned einen Speedup von bis zu 68 % erreicht (bei 1024 Prozessen).
Damit zeigt sich, dass die in [5] berichteten Speedups in diesem Setup nicht reproduziert werden konnten. Die Optimierungspotenziale von persistent bzw. partitioned MPI sind im Strong Scaling offenbar stark von der zugrunde liegenden Hardware sowie der konkreten MPI Implementierung abhängig.

Weak Scaling

Im Weak Scaling fallen die Unterschiede zwischen den Kommunikationsmustern ebenfalls gering aus. Die beobachteten Speedups von persistent und partitioned gegenüber non-blocking liegen nur im Bereich von ca. 0,2-0,9 %, sodass insgesamt nur ein schwacher Trend zugunsten der optimierten Varianten sichtbar ist.
Collom et al. [@3] berichten demgegenüber im Weak Scaling bei 4096 Prozessen Speedups von 12,5 % (persistent) bzw. 27 % (partitioned) relativ zur Baseline.
Entsprechend ist auch hier davon auszugehen, dass die Speedups gegenüber non-blocking durch die spezifische Hardwareplattform und die eingesetzte MPI Implementierung bestimmt werden.

Size-Increase

In der Size-Increase Studie wird ein stark nachrichtengrößenabhängiges Verhalten beobachtet. Bei kleinen Nachrichten zeigt sich ein deutlicher Vorteil der optimierten Varianten, während der relative Speedup mit steigender Halo-Größe abnimmt. Bei 6 KiB wird ein Maximum von ca. 128 % (partitioned) bzw. 72 % (persistent) gegenüber non-blocking beobachtet. Bei 1536 KiB konvergiert der Vorteil beider Varianten auf etwa 1,6 %.
Collom et al. [5] berichten hingegen von einem anderen Verhalten. Bei 6 KiB ist persistent etwa gleichauf mit der Baseline, während partitioned deutlich schlechter abschneidet (ca. -73 %). Mit zunehmender Nachrichtengröße kehrt sich dieses Verhältnis um. Bei 1536 KiB werden Speedups von ca. +21 % (persistent) und ca. 37 % (partitioned) gegenüber non-blocking berichtet.
Interessant ist hierbei, dass sich die beobachteten Nachrichtengrößen-Trends zwischen beiden Studien deutlich unterscheiden. In dieser Studie treten die größten Vorteile bei kleinen Nachrichten auf und nehmen mit steigender Nachrichtengröße ab, während [5] die stärksten Vorteile bei großen Nachrichten berichtet und bei kleinen Nachrichten teilweise sogar negative Speedups

gegenüber non-blocking beobachtet. Dies deutet erneut darauf hin, dass sich die Unterschiede der Kommunikationsmuster stark von der jeweiligen Hardwareumgebung sowie der konkreten MPI Implementierung beeinflussen lassen.

Threads-pro-Prozess

In der Threads-pro-Prozess Studie sind persistent und partitioned überwiegend leicht besser als non-blocking. Als Ausnahme tritt wie zuvor bereits erwähnt ein Messpunkt mit 4 Threads pro Prozess auf, in dem beide optimierten Varianten minimal negative Speedups gegenüber der Referenz zeigen.
Collom et al. [5] zeigen in ihrer durchgeführten Studie, dass persistent über alle getesteten Konfigurationen etwa 20 % schneller als die Baseline ist. Partitioned schneidet bei 64 Threads pro Prozess deutlich schlechter ab als die Baseline und persistent. Mit sinkender Threadanzahl pro MPI-Prozess überholt partitioned zunächst die Baseline und ab etwa 8 Threads pro Prozess auch persistent.
Konkrete Ursachen für die abweichenden Verläufe lassen sich aus den vorliegenden Messdaten nicht zuverlässig ableiten und würden ein detailliertes Profiling der Kommunikationspfade erfordern.

Gesamteinordnung

Außerhalb der Size-Increase Studie fallen die Speedups in [5] insgesamt deutlich größer aus als in den hier betrachteten Messungen. Besonders hervorzuheben ist jedoch, dass in der vorliegenden Size-Increase Studie bei der kleinsten Nachrichtengröße (6 KiB) ein maximaler Speedup erreicht wird, der in der Referenzarbeit in dieser Form nicht beobachtet wird.
Als plausible Erklärungsansätze kommen geringfügige Unterschiede in der Setup-Konfiguration, die verwendete MPI Implementierung (Open MPI 5.0.3 vs. MVAPICH2 2.3.7) sowie unterschiedliche Hardwareplattformen in Betracht. Ebenso nutzen Collom et al. [5] für die OpenMP-Parallelisierung explizit Multithreading, während dieses Threading-Szenario in dieser Studie nicht nachgebildet werden konnte und stattdessen eine Alternativlösung mit zwei Threads auf zwei physischen Kernen verwendet wird. Dies kann die beobachteten Abweichungen zwischen beiden Untersuchungen zusätzlich beeinflussen.

5.2 MPI-Ring-Benchmark

Da insbesondere die Size-Increase Studie stark abweicht, wird in diesem Abschnitt versucht, diese Studie in einem MPI-Ring-Benchmark nachzustellen, um den Einfluss der reinen Kommunikation stärker zu isolieren (ohne Comb-spezifische Aspekte wie Packing/Unpacking). Im Fokus steht dabei, welchen Einfluss die Amortisierung der Kommunikationsinitialisierung über die Iterationsschleife hat, da dieses Prinzip sowohl persistent als auch partitioned MPI zugrunde liegt.

5.2.1 Absolute Kommunikationszeiten

Die Messergebnisse des MPI-Ring-Benchmarks (Tabelle 5.11 und Abbildung 5.9) zeigen, dass sich non-blocking und persistent Kommunikation im absoluten Verlauf sehr ähnlich verhalten.

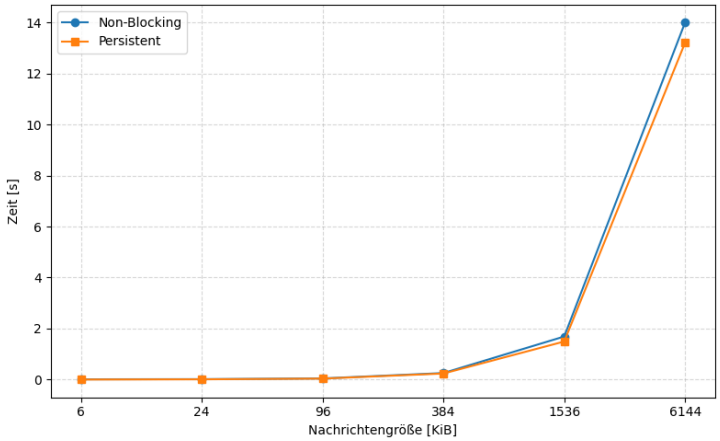


Abb. 5.9: MPI-Ring-Benchmark: Verlauf der absoluten Kommunikationszeiten

Tab. 5.11: MPI-Ring-Benchmark: Kommunikationszeiten

Größe einer Nachricht [KiB]	Non-blocking [s]	Persistent [s]
6	0,014	0,005
24	0,020	0,013
96	0,049	0,042
384	0,259	0,241
1536	1,691	1,495
6144	13,989	13,197

Im Bereich kleiner bis mittlerer Nachrichten (6 KiB bis 384 KiB) sind die Kommunikationszeiten insgesamt sehr gering ($< 0{,}3$ s). Dadurch sind die absoluten Unterschiede zwischen den Varianten in einer linear skalierten Darstellung der Kommunikationszeit optisch kaum zu erkennen und die Kurven wirken nahezu deckungsgleich. Bei größeren Nachrichten werden die absoluten Differenzen in dieser Darstellung deutlicher sichtbar, ohne dass damit zwangsläufig ein großer relativer Abstand impliziert ist.
Während die Abstände im absoluten Vergleich eher gering erscheinen, werden sie im relativen Vergleich deutlicher sichtbar.

5.2.2 Relative Kommunikationszeiten

Der relative Speedup der persistent Variante gegenüber non-blocking (Tabelle 5.12 und Abbildung 5.10) offenbart ein aufschlussreiches Muster, das die Overhead-Struktur der Kommunikationsinitialisierung deutlich macht.

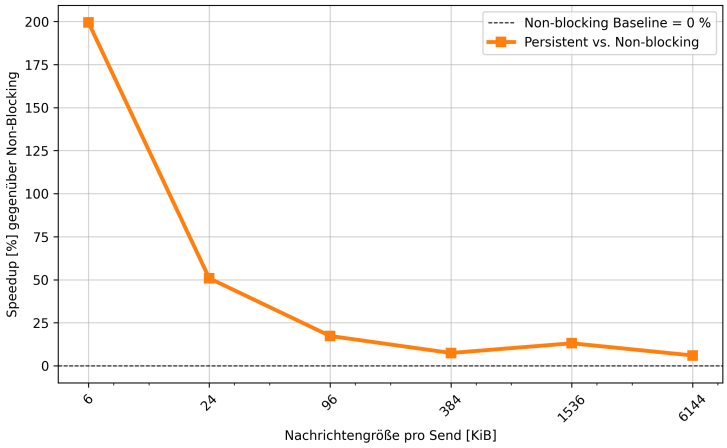


Abb. 5.10: MPI-Ring-Benchmark: Verlauf des relativen Speedups gegenüber non-blocking MPI

Tab. 5.12: MPI-Ring-Benchmark: Relativer Speedup gegenüber non-blocking MPI

Größe [KiB]	Speedup Persistent [%]
6	199,5
24	50,8
96	17,3
384	7,4
1536	13,1
6144	6,0

Extreme Gewinne bei kleinen Nachrichten Bei 6 KiB erzielt persistent Kommunikation einen Speedup von etwa 199,5 %. Dies liegt deutlich über dem entsprechenden Wert der Size-Increase Studie (72,4 % bei persistent, Unterabschnitt 5.1.3). Dieser Messpunkt verdeutlicht also auch, dass die Einsparung durch die persistent Initialisierung bei sehr kleinen Nachrichten besonders stark zum Tragen kommt.

Kontinuierlicher Abfall mit Nachrichtengröße Der Speedup nimmt kontinuierlich ab. Er liegt bei 50,8 % bei 24 KiB, 17,3 % bei 96 KiB und 7,4 % bei 384 KiB. Dieses Muster folgt der gleichen konzeptuellen Logik wie in der Size-Increase Studie. Mit zunehmender Datenmenge dominiert die Transferzeit den Gesamtaufwand, sodass softwareseitige Optimierungen weniger ins Gewicht fallen.

Bruch im Verlauf bei mittleren Nachrichten Ein Strukturbruch tritt bei 1536 KiB auf, bei dem der Speedup unerwartet auf 13,1 % ansteigt, anstatt wie erwartet weiter abzunehmen. Bei 6144 KiB sinkt er dann erneut auf 6,0 %. Diese Anomalie kann auf Effekte hindeuten, die über die reine Nachrichtengröße hinausgehen und könnte möglicherweise durch Systemrauschen oder Cache-Effekte begründet werden.

5.2.3 Validierung der Amortisierungseffekte

Wenn die Nachrichtengröße in der Kommunikation minimal ist, erreicht der Anteil der Initialisierungszeit an der Gesamtlaufzeit sein Maximum. In diesem isolierten Versuchsaufbau treten die entsprechenden Effekte sogar deutlicher hervor als in der replizierten Size-Increase Studie. Insgesamt bestätigen die Ergebnisse des MPI-Ring-Benchmarks die Annahme, dass persistent Requests insbesondere dann vorteilhaft sind, wenn der reine Datentransfer nur einen geringen Anteil an den Kommunikationskosten ausmacht und Initialisierungs-Overheads den Gesamtaufwand dominieren. Im nächsten Schritt wird dieser Zusammenhang mithilfe des RDTSC-Benchmarks im PML-UCX-Modul von Open MPI auf konzeptioneller Ebene untersucht.

5.3 RDTSC-Benchmark

Mithilfe dieses Benchmarks lassen sich nun die zentralen Unterschiede im Kontrollfluss zwischen non-blocking und persistent MPI gegenüberstellen. Die folgenden Messergebnisse in P. 5.1 fassen die durchschnittlichen CPU-Taktzyklen pro Sendeaufruf zusammen und dienen als abstrahierte Grundlage zur Einordnung des Initialisierungs-Overheads beider Kommunikationsmuster.

```
f_isend_like cycles / call                          : 8
f_start_like cycles (+ 1x f_isend_init_like) / call : 6
ratio (isend / start)                               : 1.3333
```

P. 5.1: RDTSC: Benchmark-Ergebnisse

In dieser Abstraktion ist der non-blocking Kontrollfluss pro Iteration um den Faktor $\frac{8}{6} \approx 1{,}33$ teurer als der persistent vorbereitete Kontrollfluss (also ca. 33 %).
Damit wird der erwartete Unterschied zwischen iterativer Initialisierung (non-blocking) und einmaliger Vorbereitung mit Wiederverwendung (persistent/partitioned) in diesem Modell auch sichtbar.
Gleichzeitig sind die in Unterunterabschnitt 4.4.2 diskutierten Limitationen zu berücksichtigen, sodass die erhaltenen Unterschiede in den Taktzyklen nur eingeschränkt interpretierbar sind. Die Ergebnisse sind daher primär als indikative Bestätigung der vorherigen Trends zu verstehen.

6 Fazit

6.1 Zusammenfassung der Ergebnisse

Dieser Abschnitt fasst die zentralen Ergebnisse der Arbeit zusammen und ordnet sie im Kontext der untersuchten Kommunikationsmuster ein. Dazu werden die Befunde aus den Comb-Studien, dem MPI-Ring- und dem RDTSC-Benchmark strukturiert dargestellt und zusammengeführt.

Kernergebnisse Comb-Studien Die Experimente zeigen eine insgesamt hohe Parallelisierungsfähigkeit des Comb-Benchmarks und folgen in den zugehörigen Studien den Gesetzmäßigkeiten von Amdahl und Gustafson. Einzelne superlineare Effizienztendenzen deuten auf zusätzliche Cache-Effekte, netzwerkseitige Faktoren oder generelles Systemrauschen hin. Die genauen Ursachen müssten jedoch durch umfassendes Profiling identifiziert werden.
Die relativen Unterschiede zwischen non-blocking, persistent und partitioned MPI sind häufig gering, dennoch deuten die Messergebnisse auf einen geringfügigen Performance-Vorteil der optimierten Varianten hin.
Größere Vorteile dieser moderneren Muster zeigen sich ausschließlich in der Size-Increase Studie bei kleinen Nachrichten, da hier der Initialisierungsaufwand einen relevanten Anteil an der Gesamtlaufzeit ausmacht. In diesem Fall amortisiert sich die einmalige Initialisierung über mehrere Iterationen, wobei der partitioned Datentransfer zusätzliche Vorteile gegenüber persistent MPI bietet.

Abgleich mit Collom et al. (Comb) Im Vergleich zur Referenzarbeit [5] fallen die beobachteten Speedups insgesamt kleiner aus und einzelne Trends (Size-Increase Studie) weichen sichtbar davon ab. Damit wird deutlich, dass die Vorteile der einzelnen Kommunikationsmuster stark von Hardwareplattform, MPI Implementierung und Thread-Setup abhängen können. Schlussfolgernd spiegeln die Ergebnisse vor allem die Eigenschaften des eingesetzten Systems und Software-Stacks wider, weshalb sich die Vorteile der Muster nur eingeschränkt verallgemeinern lassen.

73

MPI-Ring-Benchmark In dem isolierten Ring-Setup ist der Einfluss der amortisierten Initialisierung klar sichtbar. Der zentrale Befund ist demnach ein klarer Größeneffekt. Der Nutzen von persistent MPI ist bei kleinen Nachrichten am größten und nimmt mit wachsender Nachrichtengröße ab. Einzelne Abweichungen vom erwarteten Trend können auf system- oder netzwerkabhängige Einflüsse hindeuten.
Insgesamt stützen diese Beobachtungen die Ergebnisse der Size-Increase Studie und zeigen, dass die relativen Vorteile stark von der übertragenen Datenmenge beeinflusst werden.

RDTSC-Benchmark RDTSC misst über 1000 Iterationen einen etwa 33 % höheren Overhead pro Sendeaufruf bei der non-blocking, iterativen Initialisierung im Vergleich zur einmaligen Vorbereitung bei der persistent Variante. Da partitioned MPI die persistent Requests ebenfalls nutzt, lassen sich die beobachteten Vorteile teilweise auch auf partitioned übertragen. In Verbindung mit den Size-Increase Studien sowie dem MPI-Ring-Benchmark ergibt sich ein stimmiges Gesamtbild. Die beobachteten Performance-Vorteile gegenüber non-blocking MPI sind überwiegend auf den Amortisationseffekt zurückzuführen. Wenn der Anteil des Datentransfers über das Netzwerk an der Gesamtlaufzeit gering ist, wird das Ausmaß dieser Amortisation besonders verdeutlicht.

6.2 Ausblick

Ein logischer nächster Schritt ist ein detailliertes Profiling der Kommunikationspfade, um die in dieser Arbeit nur indirekt sichtbaren Unterschiede zwischen persistent und partitioned MPI genauer zu untersuchen. Da partitioned MPI auf persistent aufsetzt und es um das Partitionierungskonzept erweitert, ist insbesondere zu klären, in welchen Codeabschnitten zusätzlicher Nutzen oder Overhead entsteht. Dazu zählen vor allem Setup, Synchronisation und Datentransfer. Ergänzend sollten die in den Studien diskutierten möglichen Cache-Effekte durch gezieltes Hardware- und Memory-Profiling abgesichert werden. So lassen sich beobachtete (leicht) superlineare Effekte oder Abweichungen vom erwarteten Trend belastbar einordnen.
Darüber hinaus legen die Abweichungen zur Referenzarbeit [5] nahe, den Einfluss der MPI Implementierung systematischer zu untersuchen. Eine sinnvolle Folgeuntersuchung wäre, die gleichen Studien auf derselben HPC-Umgebung

mit unterschiedlichen Implementierungen zu wiederholen. So ließe sich der Einfluss der Hardware weitgehend konstant halten und der Beitrag der MPI Implementierung zu den beobachteten Trends separat bewerten. Umgekehrt könnte dieselbe MPI Implementierung auf verschiedenen Systemen untersucht werden, um die Rolle der Hardware zu kapseln.
Schließlich bietet die Thread-Konfiguration weiteres Potenzial für Folgearbeiten. In den Experimenten wurde eine feste Thread-Konfiguration verwendet, sodass der Einfluss der Thread-Beteiligung an der Kommunikation nur eingeschränkt sichtbar wird. Eine naheliegende Erweiterung wäre daher, gezielt die Anzahl der an Packing/Unpacking beteiligten Threads zu variieren (z.B. 2, 4, 8, 16). Diese Ergebnisse könnten einer Konfiguration gegenübergestellt werden, in der dieser Anteil bewusst kleiner gehalten wird. Auf diese Weise lässt sich der Effekt der prozessinternen Parallelisierung auf die Kommunikationsmuster gezielter untersuchen. In diesem Zusammenhang wäre auch eine Variation verschiedener Partitionierungs-Mappings interessant, wie es bei Schneewind und Niethammer [30] untersucht wird. Anstatt eines strikten 1:1-Mappings (eine Partition pro Thread) könnten mehrere kleine Partitionen zu größeren Einheiten zusammengefasst und erst dann übertragen werden. Somit könnten die Auswirkungen verschiedener Mapping-Strategien auf die Performance von partitioned MPI im Comb-Benchmark detaillierter untersucht werden.

Literatur

[1]Gene M. Amdahl. “Validity of the single processor approach to achieving large scale computing capabilities”. In: AFIPS ’67 (Spring). Atlantic City, New Jersey: Association for Computing Machinery, 1967, S. 483–485 (zitiert auf den Seiten 42, 57).

[2]Peter J Braam und Rumi Zahir. “Lustre: A scalable, high performance file system”. In: *Cluster File Systems, Inc* 8.11 (2002), S. 3429–3441 (zitiert auf Seite 4).

[5]Gerald Collom, Jason Burmark, Olga Pearce und Amanda Bienz. “Persistent and Partitioned MPI for Stencil Communication”. In: *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 2024, S. 1–7 (zitiert auf den Seiten iv, 2, 3, 14, 19, 22, 23, 25, 28, 30, 36, 37, 41–43, 45, 56, 66–68, 73, 74).

[6]L. Dagum und R. Menon. “OpenMP: an industry standard API for shared-memory programming”. In: *IEEE Computational Science and Engineering* 5.1 (1998), S. 46–55 (zitiert auf Seite 4).

[7]Dario Dematties, George Thiruvathukal, Silvio Rizzi, Alejandro Wainselboim und B Zanutto. “Towards High-End Scalability on Biologically-Inspired Computational Models”. In: *Parallel Computing: Technology Trends*. Bd. 36. IOS Press Books, März 2020, S. 497–506 (zitiert auf Seite 44).

[8]D L Eager, J Zahorjan und E D Lazowska. “Speedup versus efficiency in parallel systems”. In: *IEEE Transactions on Computers* 38.3 (März 1989), S. 408–423 (zitiert auf Seite 44).

[9]Edgar Gabriel, Graham E. Fagg, George Bosilca et al. “Open MPI: Goals, Concept, and Design of a Next Generation MPI Implementation”. In: *Recent Advances in Parallel Virtual Machine and Message Passing Interface*. Hrsg. von Dieter Kranzlmüller, Péter Kacsuk und Jack Dongarra. Springer Berlin Heidelberg, 2004, S. 97–104 (zitiert auf Seite 5).

[10]William Gropp, Ewing Lusk, Nathan Doss und Anthony Skjellum. “A high-performance, portable implementation of the MPI message passing interface standard”. In: *Parallel Computing* 22.6 (1996), S. 789–828 (zitiert auf Seite 5).

[11]John Gustafson. “Reevaluating Amdahl’s Law”. In: *Commun. ACM* 31 (Mai 1988), S. 532–533 (zitiert auf Seite 42).

[12]Yiltan Hassan Temucin, Ryan E. Grant und Ahmad Afsahi. “Micro-Benchmarking MPI Partitioned Point-to-Point Communication”. In: *Proceedings of the 51st International Conference on Parallel Processing*. ICPP ’22. Bordeaux, France: Association for Computing Machinery, 2023 (zitiert auf den Seiten 15, 20, 23).

[13]Hewlett Packard Enterprise (HPE). *Cray MPICH Documentation (HPE Cray Programming Environment v24.03)*. Dokumentation. Hewlett Packard Enterprise (HPE). 2024 (zitiert auf Seite 6).

[14]Intel Corporation. *Intel MPI Library: Developer Reference for Linux OS*. Dokumentation (Version 2021.16). Intel Corporation. 2021 (zitiert auf Seite 5).

[15]James Muir. *Using the RDTSC Instruction for Performance Monitoring*. Jan. 1998 (zitiert auf Seite 50).

[16]Christoph Kessler und Jörg Keller. “Models for Parallel Computing: Review and Perspectives”. In: *Mitteilungen-Gesellschaft für Informatik eV, Parallel-Algorithmen und Rechnerstrukturen* 24 (Dez. 2007), S. 13–29 (zitiert auf Seite 6).

[17]Fredrik Berg Kjolstad und Marc Snir. “Ghost Cell Pattern”. In: *Proceedings of the 2010 Workshop on Parallel Programming Patterns*. ParaPLoP ’10. Carefree, Arizona, USA: Association for Computing Machinery, 2010 (zitiert auf den Seiten 17–19).

[21]Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard Version 4.0*. Juni 2021 (zitiert auf den Seiten 2, 5–7, 11, 13, 38).

[22]MVAPICH Team. *MVAPICH2 2.3.6 User Guide*. Dokumentation. Network-Based Computing Laboratory (NBCL), Department of Computer Science und Engineering, The Ohio State University. Mai 2021 (zitiert auf Seite 5).

[24]Gregory F Pfister. “An introduction to the infiniband architecture”. In: *High performance mass storage and parallel I/O* 42.617-632 (2001), S. 102 (zitiert auf Seite 4).

[26]Rolf Rabenseifner, Georg Hager und Gabriele Jost. “Hybrid MPI/OpenMP Parallel Programming on Clusters of Multi-Core SMP Nodes”. In: *Proceedings of the 2009 17th Euromicro International Conference on Parallel, Distributed and Network-Based Processing*. PDP ’09. USA: IEEE Computer Society, 2009, S. 427–436 (zitiert auf Seite 4).

[27]Sasko Ristov, Radu Prodan, Marjan Gusev und Karolj Skala. “Superlinear speedup in HPC systems: Why and when?” In: *2016 Federated Conference on Computer Science and Information Systems (FedCSIS)*. IEEE. 2016, S. 889–898 (zitiert auf den Seiten 44, 60).

[29]Frank Schmuck und Roger Haskin. “GPFS: A Shared-Disk File System for Large Computing Clusters”. In: *FAST’02: Proceedings of the 1st USENIX conference on File and storage technologies*. USENIX Association, 2002, S. 16 (zitiert auf Seite 4).

[30]Axel Schneewind und Christoph Niethammer. *Benchmarking the State of MPI Partitioned Communication in Open MPI*. Short Paper. Sep. 2024 (zitiert auf den Seiten 21, 23, 75).

[31]Pavel Shamis, Manjunath Gorentla Venkata, M. Graham Lopez et al. “UCX: An Open Source Framework for HPC Network APIs and Beyond”. In: *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. 2015, S. 40–43 (zitiert auf den Seiten 16, 51).

[33]Marco Verdicchio und Carlos Teijeiro Barjas. “Introduction to High-Performance Computing”. In: *High Performance Computing for Drug Discovery and Biomedicine*. Hrsg. von Alexander Heifetz. New York, NY: Springer US, 2024, S. 15–29 (zitiert auf Seite 4).

[34]H. Wan, M. A. Giorgetta, G. Zängl et al. “The ICON-1.2 hydrostatic atmospheric dynamical core on triangular grids – Part 1: Formulation and performance of the baseline version”. In: *Geoscientific Model Development* 6.3 (2013), S. 735–763 (zitiert auf Seite 1).

[35]Andy B. Yoo, Morris A. Jette und Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Workshop on job scheduling strategies for parallel processing*. Hrsg. von Dror Feitelson, Larry Rudolph und Uwe Schwiegelshohn. Springer Berlin Heidelberg, 2003, "44–60 (zitiert auf Seite 4).

Webpages

[@3]Jason Burmark. *Comb*. develop branch, commit: e0a74ee. Lawrence Livermore National Lab (LLNL). Feb. 2023. URL: `https://github.com/llnl/Comb/tree/develop` (besucht am 22. Jan. 2026) (zitiert auf den Seiten iv, 2, 22, 24, 30, 67).

[@4]c9x.me. *RDTSC: Read Time-Stamp Counter (x86 Instruction Set Reference)*. URL: `https://c9x.me/x86/html/file_module_x86_id_278.html` (besucht am 7. Feb. 2026) (zitiert auf Seite 50).

[@18]Lawrence Livermore National Laboratory (LLNL). *mpiP*. main branch, commit: 8ff38c3. Jan. 2026. URL: `https://github.com/LLNL/mpiP/tree/main` (besucht am 17. Jan. 2026) (zitiert auf Seite 28).

[@19]Xin Li. *Scalability: strong and weak scaling*. PDC Blog, KTH Royal Institute of Technology. Nov. 2018. URL: `https://www.kth.se/blogs/pdc/2018/11/scalability-strong-and-weak-scaling/` (besucht am 27. Jan. 2026) (zitiert auf Seite 42).

[@20]Message Passing Interface Forum. *MPI Implementation Status*. 2026. URL: `https://www.mpi-forum.org/implementation-status` (besucht am 14. Jan. 2026) (zitiert auf Seite 5).

[@23]Open MPI Project. *pml_ucx.c (Open MPI: PML/UCX component source file)*. v5.0.x branch, commit: 8cc0d97. Jan. 2026. URL: `https://github.com/open-mpi/ompi/blob/v5.0.x/ompi/mca/pml/ucx/pml_ucx.c` (besucht am 24. Jan. 2026) (zitiert auf den Seiten 51, 52).

[@25]Open MPI Project. *Issue No. 12969: Partitioned communication with multiple neighbors*. Dez. 2024. URL: https://github.com/open-mpi/ompi/issues/12969 (besucht am 24. Jan. 2026) (zitiert auf Seite 49).

[@28]RookieHPC. *MPI Exercises. Praxisübungen zur MPI-Programmierung*. URL: https://rookiehpc.org/mpi/exercises/ (besucht am 22. Jan. 2026) (zitiert auf Seite 7).

[@32]user28656646. *OpenMpi5 Partitioned Communication with multiple neighbors*. 6. Dez. 2024. URL: https://stackoverflow.com/questions/79258925/openmpi5-partitioned-communication-with-multiple-neighbors (besucht am 24. Jan. 2026) (zitiert auf Seite 49).

[@36]Johannes Gutenberg-Universität Mainz Zentrum für Datenverarbeitung. *Compute Nodes*. Hardware Spezifikationen des Mogon Clusters. 2026. URL: https://docs.hpc.uni-mainz.de/docs/cluster/compute-nodes/ (besucht am 27. Jan. 2026) (zitiert auf den Seiten 6, 40).

Abbildungsverzeichnis

2.1 Schematischer Aufbau eines HPC-Clusters 5
2.2 traditioneller vs. partitioned Sendeverlauf (angelehnt an [12]) . . . 15
2.3 5-Punkt-Stencil [17] 17
2.4 Halo-Austausch zweier Subdomänen 5-Punkt-Stencil [17] 18
2.5 Halo-Austausch für den 9-Punkt-Stencil [17] 19

4.1 Comb: Flächenaustausch 26
4.2 Comb: Kantenaustausch 26
4.3 Comb: Eckpunktaustausch 27
4.4 Comb: Gesamtszenario 27
4.5 Ordnerstruktur des Comb-Benchmarks 30
4.6 Interne Datenstruktur der Halo-Nachrichten 33
4.7 PML-Layer als Vermittlungsschicht (angelehnt an [31]) 51

5.1 Strong Scaling: Verlauf der absoluten Kommunikationszeiten 56
5.2 Strong Scaling: Verlauf der relativen Speedups gegenüber non-blocking MPI 58
5.3 Weak Scaling: Verlauf der absoluten Kommunikationszeiten 59
5.4 Weak Scaling: Verlauf der relativen Speedups gegenüber non-blocking MPI 61
5.5 Size-Increase: Verlauf der absoluten Kommunikationszeiten 62
5.6 Size-Increase: Verlauf der relativen Speedups gegenüber non-blocking MPI 63
5.7 Threads-pro-Prozess: Verlauf der absoluten Kommunikationszeiten 64
5.8 Threads-pro-Prozess: Verlauf der relativen Speedups gegenüber non-blocking MPI 65
5.9 MPI-Ring-Benchmark: Verlauf der absoluten Kommunikationszeiten 69
5.10 MPI-Ring-Benchmark: Verlauf des relativen Speedups gegenüber non-blocking MPI 70

Tabellenverzeichnis

2.1 Parameter der Funktion `MPI_Send` 9

4.1 Comb: geometrische Zusammensetzung der Halo-Flächen 26
4.2 Callsite Message Sent statistics (all, sent bytes) 29

5.1 Strong Scaling: Kommunikationszeiten 57
5.2 Strong Scaling: Effizienz (Referenz: non-blocking MPI) 57
5.3 Strong Scaling: Relative Speedups gegenüber non-blocking MPI . . 58
5.4 Weak Scaling: Kommunikationszeiten 60
5.5 Weak Scaling: Effizienz (Referenz: non-blocking MPI) 60
5.6 Weak Scaling: Relative Speedups gegenüber non-blocking MPI . . . 61
5.7 Size-Increase: Kommunikationszeiten 62
5.8 Size-Increase: Relative Speedups gegenüber non-blocking MPI . . . 63
5.9 Threads-pro-Prozess: Kommunikationszeiten 65
5.10 Threads-pro-Prozess: Relative Speedups gegenüber non-blocking MPI 65
5.11 MPI-Ring-Benchmark: Kommunikationszeiten 69
5.12 MPI-Ring-Benchmark: Relativer Speedup gegenüber non-blocking MPI 70

81

Programmausdrucksverzeichnis

2.1 Blocking `Send/Recv` 8
2.2 Non-blocking Send/Recv mit Wait 9
2.3 Persistent MPI mit Schleifen 11
2.4 Partitioned MPI mit MPI_Init_thread und OpenMP-Threads 13

4.1 Vereinfachte Darstellung der partitioned Setup-Schleifen 32
4.2 Partitioned `pack`: Gitterarray → Puffer + `Pready` pro Partition . . . 34
4.3 Partitioned `unpack`: Puffer → Gitterarray 34
4.4 Persistent Kommunikationsreihenfolge: pack → Isend 37
4.5 Partitioned Kommunikationsreihenfolge: Isend → pack 38
4.6 MPI-Umgebung initialisieren mit `MPI_THREAD_FUNNELED` 39
4.7 MPI-Umgebung initialisieren mit `MPI_THREAD_MULTIPLE` 39
4.8 MPI-Ring-Benchmark: Initialisierung und Topologie 47
4.9 MPI-Ring-Benchmark: Puffer-Management 47
4.10 MPI-Ring-Benchmark: non-blocking Kommunikation 48
4.11 MPI-Ring-Benchmark: persistent Kommunikation 48
4.12 RDTSC: f_isend_like 52
4.13 RDTSC: f_isend_init_like 52
4.14 RDTSC: f_start_like 53
4.15 RDTSC: Messung über Inline-Assembly 53
4.16 RDTSC: Kompilierung des Benchmarks 55

5.1 RDTSC: Benchmark-Ergebnisse 72

Kolophon

Diese Arbeit wurde mit LaTeX 2_ε gesetzt. Sie verwendet den von Ricardo Langner entwickelten Stil *Clean Thesis*. Das Design des Stils *Clean Thesis* ist von Benutzerhandbüchern von Apple Inc. inspiriert.

Laden Sie den Stil *Clean Thesis* unter `http://cleanthesis.der-ric.de/` herunter.

Anpassungen an den Stil des Instituts für Informatik sind unter `https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu` zu finden.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen[1] stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden[2]. Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende

[1] Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

[2] Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, 10. Februar 2026

L. Emter

Leonard Emter

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Nutzung von KI-Tools.

KI Tool	Genutzt für	Grund	Wann
Perplexity	Suche relevanter Literatur	Literaturrecherche	Abschnitt 1.1, Abschnitt 2.5, Kapitel 3
ChatGPT	Unterstützung bei Erstellen von Debug-Prints	Fehlerbehebung bei partitioned Implementierung	Unterabschnitt 4.1.2
ChatGPT	LaTeX-Hinweise	Korrekte Formatierung, Kennenlernen von Befehlen	Alle Kapitel
DeepL Write	Neuformulierung meiner Textentwürfe. Rechtschreib- und Grammatikkontrolle	Bessere Lesbarkeit	Alle Kapitel
ChatGPT	Unterstützung in bash-Skripten	Benchmark-Ergebnisse extrahieren und strukturiert aufbereiten	Abschnitt 5.1
ChatGPT	Unterstützung in Python-Plotting Skripten	Benchmark-Ergebnisse korrekt extrahieren und visualisieren	Kapitel 5